



UNIVERSIDAD PONTIFICIA DE SALAMANCA
FACULTAD DE INFORMÁTICA
Grado en Informática

Trabajo Fin de Grado

Estudio sobre Google Kubernetes Engine (GKE): diseño, implementación y validación de arquitecturas cloud-native

Víctor García Crespo

Roberto Berjón Gallinas
Director

Salamanca, junio de 2026

Resumen

Este proyecto presenta un estudio teórico-práctico sobre el diseño y despliegue de una arquitectura cloud-native basada en microservicios, utilizando Google Kubernetes Engine (GKE) como plataforma de orquestación. El objetivo principal ha sido validar cómo las tecnologías en la nube resuelven los retos de la ingeniería de software actual, garantizando la alta disponibilidad, la resiliencia y la automatización total de los despliegues.

Para ello, se ha desarrollado un backend en Java con el framework Quarkus, conectado a una base de datos PostgreSQL gestionada mediante Cloud SQL. La solución se ha empaquetado con Docker y desplegado en un clúster de Kubernetes, configurando un controlador Ingress para la gestión del tráfico externo y un Horizontal Pod Autoscaler (HPA) para dotar al sistema de elasticidad ante variaciones en la carga de trabajo.

Finalmente, la arquitectura se ha sometido a un riguroso plan de pruebas de estrés y de tolerancia a fallos. Los resultados validan que el clúster es capaz de autorrepararse (self-healing), realizar actualizaciones sin interrupciones (rolling updates) y absorber picos de tráfico de forma dinámica. En conclusión, este trabajo demuestra que la adopción de un enfoque cloud-native optimiza tanto la fiabilidad del servicio como la eficiencia en el uso de recursos físicos en entornos de producción.

Abstract

This project presents a comprehensive theoretical and practical study of the design and deployment of a cloud-native microservices architecture, using Google Kubernetes Engine (GKE) as the primary orchestration platform. The core objective was to validate that modern cloud technologies effectively address contemporary software engineering challenges by guaranteeing high availability, systemic resilience, and fully automated deployment pipelines.

To accomplish this, a Java-based backend was engineered using the Quarkus framework and integrated with a PostgreSQL database managed via Google Cloud SQL. The resulting solution was containerized with Docker and deployed across a Kubernetes cluster. The infrastructure incorporates an Ingress controller for external traffic routing and a Horizontal Pod Autoscaler (HPA) to ensure elasticity under fluctuating workloads.

Finally, the architecture underwent rigorous stress and fault-tolerance testing. The empirical results confirm the cluster's capacity for autonomous self-healing, zero-downtime rolling updates, and dynamic absorption of traffic spikes. Ultimately, this research demonstrates that adopting a cloud-native paradigm significantly optimizes both service reliability and the efficient utilization of physical resources in production environments.

Descriptores

Cloud-native, Microservicios, Kubernetes, Autoescalado, Docker.

Índice

1.	Introducción	1
1.1	Motivación y Contexto.....	1
1.2	Objetivos del proyecto.....	1
1.3	Metodología y Planificación.....	2
2.	Marco Teórico y Tecnológico.....	3
2.1	Contexto y Evolución de las Arquitecturas	3
2.1.1	Evolución de las Arquitecturas: De Monolitos a Microservicios.....	3
2.1.2	Tecnologías de Virtualización y Contenedores (Docker)	4
2.1.3	El paradigma de la Orquestación de Contenedores	4
2.2	Conceptos Fundamentales y Origen	4
2.3	El Paradigma de la Orquestación Declarativa	5
2.4	Arquitectura Interna del Cluster	6
2.4.1	Plano de Control (Control Plane).....	6
2.4.2	Nodos de Trabajo (Worker Nodes).....	7
2.5	Entidades y Recursos de la API	8
2.5.1	Pods: La unidad mínima de despliegue	8
2.5.2	Controladores: Deployments y ReplicaSets.....	9
2.5.3	Redes y Exposición: Services e Ingress	10
2.5.4	Almacenamiento y Estado: Persistent Volumes	12
2.5.5	Configuración y Seguridad.....	13
2.6	Despliegue en la Nube: IaaS frente a CaaS.....	14
2.6.1	Despliegue manual: Infraestructura como Servicio (IaaS).....	14
2.6.2	Delegar la complejidad: Contenedores como Servicio (CaaS).....	14
2.6.3	Conclusión arquitectónica y decisión del proyecto	15
2.7	Análisis Comparativo (Kubernetes vs Docker Swarm)	15
2.7.1	Filosofía de Diseño y Arquitectura Subyacente	15
2.7.2	Gestión del Clúster, Escalabilidad y Rendimiento	16
2.7.3	Tolerancia a Fallos y Alta Disponibilidad	16
2.7.4	Gestión de Redes y Almacenamiento Persistente	17
2.7.5	Ecosistema, Extensibilidad y Curva de Aprendizaje.....	18
2.7.6	Conclusión: Justificación Tecnológica de la Elección	19
3.	Especificación Requisitos.....	21
3.1	Visión General del Sistema	21
3.2	Requisitos Funcionales.....	21

3.2.1	Módulo de Catálogo de Eventos (Microservicio ws-catalogo).....	21
3.2.2	Módulo de Reservas (Microservicio ws-reservas)	22
3.2.3	Reglas de Negocio	22
3.3	Diseño y Especificación de la API REST.....	22
3.3.1	Endpoints del Catálogo de Eventos	22
3.3.2	Interfaces de Gestión de Reservas.....	23
4.	Desarrollo e Implementación	25
4.1	Metodología de Desarrollo	25
4.2	Arquitectura del Sistema.....	26
4.2.1	Seguridad y Patrón Sidecar	27
4.3	Justificación del Stack Tecnológico	27
4.3.1	Framework de Desarrollo: Quarkus.....	27
4.3.2	Contenerización: Docker.....	28
4.3.3	Orquestación y Cloud: Kubernetes y Google Kubernetes Engine (GKE)	28
4.3.4	Persistencia y Gestión de Base de Datos	28
4.3.5	Elección del Proveedor Cloud: Google Cloud Platform (GCP).....	28
4.3.6	Control de Versiones y Repositorio: Git y GitHub.....	29
4.4	Fases de Despliegue e Integración	29
4.4.1	Fase 1: Despliegue Base y Primer Microservicio.....	29
4.4.2	Fase 2: Evolución a Arquitectura Distribuida	40
4.4.3	Evolución hacia la Producción: Unificación mediante Ingress Controller	47
4.5	Plan de Pruebas y Validación de la Arquitectura	49
4.5.1	Estrategia y Entorno de Pruebas.....	49
4.5.2	Pruebas de Resiliencia y Alta Disponibilidad (Self-Healing)	50
4.5.3	Pruebas de Actualización sin Corte de Servicio (Rolling Updates).....	52
4.5.4	Pruebas de escalabilidad Automática (Autoscaling)	53
5.	Conclusiones y Trabajos Futuros	59
5.1	Objetivos alcanzados.....	59
5.2	Valoración técnica del proyecto.....	59
5.3	Trabajos Futuros	60
5.3.1	Integración avanzada de Inteligencia Artificial mediante Model Context Protocol (MCP).....	60
5.3.2	FinOps y Sostenibilidad (Green Computing)	61
5.3.3	Evolución hacia una Malla de Servicios (Service Mesh).....	61
5.3.4	Automatización total del despliegue mediante GitOps	61
5.4	Reflexión Final	61

Índice de Figuras

Figura 2-1 Comparativa entre la arquitectura monolítica y arquitectura de microservicios. .	3
Figura 2-2 Arquitectura de un clúster de Kubernetes y componentes principales.	6
Figura 2-3 Jerarquía de controladores lógicos: El Deployment administra el ReplicaSet, que a su vez garantiza la ejecución continua de los Pods.	9
Figura 2-4 Flujo de tráfico de red en Kubernetes: enrutamiento externo mediante Ingress al Service y balanceo de carga final hacia los Pods.	11
Figura 2-5 Desacoplamiento del almacenamiento en Kubernetes mediante PV y PVC.	12
Figura 4-1 Diagrama de arquitectura para la plataforma de gestión de eventos	27
Figura 4-2 Activación del clúster local de Kubernetes integrado en Docker Desktop.....	30
Figura 4-3 Despliegue exitoso de la infraestructura con Kubernetes y estado final de los Pods en el entorno de pruebas local.	33
Figura 4-4 Configuración zonal del clúster en la región de Madrid (europe-southwest1-a). 34	
Figura 4-5 Configuración de los nodos de Google Kubernetes Engine.....	34
Figura 4-6 Resumen de configuración de la instancia de Cloud SQL.....	36
Figura 4-7 . Verificación en tiempo real del arranque del Pod.....	39
Figura 4-8 Asignación de dirección IP pública al microservicio mediante el balanceador de carga de Google Kubernetes Engine.....	39
Figura 4-9 Interfaz Swagger UI del microservicio de catálogo accesible públicamente.....	40
Figura 4-10 Consola de Google Cloud SQL mostrando las bases de datos lógicas de cada microservicio coexistiendo en la misma instancia gestion-eventos.....	41
Figura 4-11 Estado de los Pods (aplicación + proxy sidecar)	45
Figura 4-12 Asignación de IP pública mediante Load Balancer	45
Figura 4-13 Interfaz interactiva de Swagger UI para el microservicio ws-catalogo desplegado en GKE	45
Figura 4-14 Interfaz interactiva de Swagger UI para el microservicio ws-reservas desplegado en GKE	46
Figura 4-15 Prueba de integración demostrando la comunicación síncrona entre el microservicio de reservas y el de catálogo a través del DNS interno de Kubernetes.....	47
Figura 4-16 Estado de los servicios del clúster tras la migración a topología ClusterIP	48
Figura 4-17 Inspección detallada del Ingress Controller en GKE	49
Figura 4-18 Estado inicial de los Pods del clúster	51
Figura 4-19 Inyección de fallo mediante la eliminación forzosa de la réplica	51
Figura 4-20 Trazabilidad de la autorreparación (Self-Healing) del Pod ws-catalogo.....	51
Figura 4-21 Estado final de los Pods del clúster	52
Figura 4-22 Estado inicial de los Pods del clúster	52
Figura 4-23 Ejecución del comando de reinicio progresivo del Deployment	52
Figura 4-24 Trazabilidad del proceso de sustitución de los Pods en tiempo real.....	53
Figura 4-25 Estado final estabilizado tras la actualización	53
Figura 4-26 Ejecución del comando de prueba de carga con ApacheBench hacia la nueva IP pública	55
Figura 4-27 Monitorización en vivo del HPA reaccionando a la subida de CPU	55
Figura 4-28 Creación secuencial de las nuevas réplicas en el nodo de Google Cloud.....	56
Figura 4-29 Estado final del clúster estabilizado con 5 réplicas	57

Índice de Tablas

Tabla 2-1 Resumen comparativo de características operativas entre Docker Swarm y Kubernetes.	19
---	----

1. Introducción

1.1 Motivación y Contexto

La transición de sistemas monolíticos a arquitecturas orientadas a microservicios responde a la necesidad industrial de construir aplicaciones escalables y tolerantes a fallos. Frente al acoplamiento inherente de los monolitos, el paradigma de microservicios aboga por la distribución de la lógica de negocio en unidades de despliegue funcionalmente independientes, lo que facilita la integración continua y la resiliencia global del sistema.

Esta evolución ha sido posible gracias a la popularización de la contenerización, con Docker como principal exponente. Sin embargo, el despliegue de microservicios a gran escala en entornos de producción introduce una complejidad crítica. La gestión manual de cientos o miles de contenedores resulta inasumible para los equipos de operaciones. Surge así la necesidad de sistemas de orquestación robustos.

En este contexto, la motivación del presente proyecto surge de la necesidad imperativa de dominar los orquestadores de contenedores que hacen posible el modelo Cloud Native. Específicamente, el trabajo se centra en el estudio de Kubernetes, una plataforma diseñada para automatizar el escalado horizontal, la gestión del estado y la autorreparación en infraestructuras distribuidas. El dominio arquitectónico de esta tecnología resulta crítico en la ingeniería actual para garantizar la alta disponibilidad en entornos productivos.

1.2 Objetivos del proyecto.

El objetivo central del trabajo radica en la investigación exhaustiva de la arquitectura interna de Kubernetes y en la evaluación de su viabilidad para orquestar servicios distribuidos con alta disponibilidad en infraestructuras en la nube.

Objetivo General:

- Diseñar, aprovisionar y validar un clúster de Kubernetes en un entorno cloud público, estableciendo una arquitectura base que garantice la escalabilidad y la disponibilidad continua de una plataforma particionada en microservicios.

Objetivos Específicos:

- Analizar el estado actual de la tecnología de contenedores y la necesidad de sistemas de orquestación en entornos profesionales.
- Realizar un estudio exhaustivo de la arquitectura interna de Kubernetes y de sus componentes principales.

- Llevar a cabo una comparativa técnica entre Docker Swarm y Kubernetes para justificar la elección tecnológica.
- Diseñar y configurar los manifiestos de Kubernetes necesarios para el despliegue de una aplicación piloto.
- Validar la solución en un entorno de nube real (GCP) mediante pruebas de tolerancia a fallos y de autoescalado.

1.3 Metodología y Planificación

Para la realización de este proyecto se ha definido una estructuración en cinco fases claramente diferenciadas siguiendo una metodología de aprendizaje progresivo y desarrollo técnico:

1. **Investigación y Estudio Teórico:** Análisis profundo del estado del arte y de la arquitectura interna de los orquestadores.
2. **Análisis y Diseño de la Solución:** Definición técnica de la arquitectura de microservicios y selección del proveedor de Cloud.
3. **Implementación y Prueba de Concepto:** Creación de imágenes Docker y configuración de manifiestos YAML para el despliegue.
4. **Experimentación y Validación:** Ejecución de pruebas de rendimiento, self-healing y escalabilidad para contrastar la teoría con la práctica.
5. **Documentación y Redacción:** Elaboración de la memoria final y preparación de la defensa.

2. Marco Teórico y Tecnológico

2.1 Contexto y Evolución de las Arquitecturas

El desarrollo de aplicaciones cloud-native ha cambiado por completo la forma en que diseñamos y desplegamos el software. Hoy en día, la industria exige sistemas que escalen rápido, que no se caigan ante fallos y que permitan subir cambios a producción de forma ágil. Para entender por qué hemos llegado a la necesidad de usar orquestadores como Kubernetes, primero hay que contextualizar por qué dejamos atrás los monolitos y cómo la contenedorización se convirtió en el estándar.

2.1.1 Evolución de las Arquitecturas: De Monolitos a Microservicios

Tradicionalmente, las aplicaciones se programaban según un patrón monolítico. En este modelo, la interfaz, la lógica de negocio y el acceso a datos están integrados en un único bloque y comparten el mismo código. Aunque esto facilita el desarrollo al principio, escalar un monolito es muy ineficiente y cualquier pequeño cambio obliga a redesplegar todo el sistema de golpe, con un alto riesgo de caída.

Para solucionar este cuello de botella, la industria adoptó la arquitectura de microservicios

- (1). La idea central es dividir una aplicación compleja en servicios pequeños, autónomos y débilmente acoplados, en los que cada uno se centra en una capacidad de negocio específica
- (2). Así, diferentes equipos pueden trabajar simultáneamente con distintas tecnologías. Además, ante un pico de tráfico, el sistema permite escalar de forma granular únicamente el microservicio que está sufriendo la carga, lo que mejora drásticamente la agilidad y la resiliencia del software.

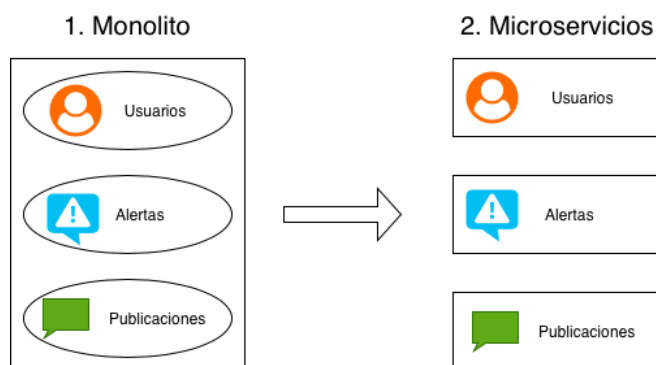


Figura 2-1 Comparativa entre la arquitectura monolítica y arquitectura de microservicios.

2.1.2 Tecnologías de Virtualización y Contenedores (Docker)

La infraestructura necesaria para soportar estas arquitecturas ha evolucionado en paralelo. Inicialmente, la virtualización tradicional mediante máquinas virtuales (VM) permitió ejecutar múltiples sistemas operativos aislados sobre un mismo hardware físico. El gran problema operativo de este enfoque es que cada VM debe incluir una copia completa de un sistema operativo invitado, lo que implica un consumo elevado de recursos (CPU, RAM y almacenamiento) y tiempos de arranque de varios minutos (3).

Aquí es donde entra Docker, marcando el inicio de la virtualización ligera. A diferencia de las VM, los contenedores comparten el núcleo del sistema operativo de la máquina anfitriona y aíslan las aplicaciones a nivel de proceso. Esto hace que sean extremadamente ligeros y portátiles, y que su arranque sea casi instantáneo. Además, Docker proporciona un estándar para empaquetar una aplicación con todas sus dependencias (binarios, librerías y configuración) en un artefacto inmutable y versionable denominado imagen. Es a partir de esta imagen de la que se instancian los contenedores en tiempo de ejecución, lo cual garantiza el principio de "escribir una vez, ejecutar en cualquier lugar", fundamental para la consistencia entre los entornos de desarrollo y producción (4).

2.1.3 El paradigma de la Orquestación de Contenedores

A pesar de las ventajas de los contenedores, su amplio uso en arquitecturas de microservicios introduce una complejidad operativa enorme. Gestionar manualmente el ciclo de vida, la red, el almacenamiento y la salud de cientos de contenedores distribuidos en múltiples servidores es un proceso insostenible a gran escala.

En este punto surge el paradigma de la orquestación. Un orquestador es un sistema diseñado para automatizar la gestión de contenedores a escala, encargándose de tareas críticas como el despliegue dinámico (scheduling), la monitorización de la salud, el auto-escalado basado en la demanda, el descubrimiento de servicios y el balanceo de carga (5). La orquestación permite transformar un conjunto de recursos de computación (nodos) en una plataforma unificada y elástica capaz de soportar aplicaciones con alta disponibilidad. Es precisamente para gobernar esta complejidad que Kubernetes se ha consolidado como el estándar indiscutible de la industria.

2.2 Conceptos Fundamentales y Origen

Una vez analizados los retos operativos que plantean los microservicios y los contenedores, queda claro que gestionar estas infraestructuras manualmente resulta inviable. Resulta indispensable contar con un "director de orquesta" que automatice todo el proceso. Aquí es donde entra Kubernetes para resolver este problema.

Kubernetes, habitualmente abreviado como K8s, es una plataforma open source, portable y extensible, diseñada para automatizar el despliegue, el escalado y la operación de aplicaciones contenedorizadas (6). En lugar de obligar al administrador a gestionar servidores individuales, esta tecnología abstrae la infraestructura física y se encarga de que todo el sistema funcione como un bloque unificado, consolidándose como el motor principal del paradigma Cloud Native.

El origen de esta herramienta se remonta a los laboratorios de Google. Kubernetes no nació desde cero, sino que fue el resultado de la experiencia de la compañía en la operación de sistemas distribuidos a escala global mediante su orquestador privativo, conocido como Borg

(7). En 2015, Google liberó el proyecto y lo donó a la Cloud Native Computing Foundation (CNCF). Desde entonces, esta fundación lidera su desarrollo de forma abierta, garantizando que sea una solución neutral y compatible con cualquier proveedor en la nube.

Hoy en día, implementar Kubernetes en entornos productivos permite dotar a las aplicaciones de capacidades críticas de resiliencia empresarial, autoescalado dinámico según la demanda de tráfico, autorreparación de contenedores fallidos (self-healing) y un balanceo de carga altamente eficiente.

2.3 El Paradigma de la Orquestación Declarativa

El pilar fundamental que diferencia a Kubernetes de otros sistemas de gestión de infraestructura es su modelo de operación basado estrictamente en la configuración declarativa. En la computación tradicional o imperativa, el administrador debe definir una secuencia exacta de comandos y pasos para alcanzar un estado. En cambio, en el modelo declarativo de Kubernetes, el ingeniero de plataforma simplemente define el estado deseado del sistema y es el propio orquestador quien asume la responsabilidad computacional de alcanzarlo y mantenerlo a lo largo del tiempo (8).

Este paradigma se materializa a nivel de la arquitectura mediante lo que se conoce como el Bucle de Control o Bucle de Reconciliación. Este proceso subyacente es un ciclo de ejecución continua que sigue tres fases críticas (9):

- **Observar:** El plano de control monitoriza en tiempo real el estado de los componentes del clúster (por ejemplo, cuántas réplicas de un microservicio están en ejecución y el consumo de recursos de los nodos).
- **Analizar/Comparar:** El sistema calcula la diferencia entre el estado real observado de la infraestructura y el estado deseado definido por el usuario mediante los manifiestos de configuración (habitualmente en formato YAML).
- **Actuar:** Si se detecta una divergencia de estado (por ejemplo, el manifiesto define cuatro réplicas para un microservicio, pero el sistema observa solo tres operativas tras la caída de un nodo), Kubernetes ejecuta de forma autónoma las operaciones correspondientes a través de su API.

Esta capacidad de corrección continua es la que permite la propiedad de autorreparación, conocida técnicamente como self-healing. Gracias a ella, las aplicaciones desplegadas se vuelven intrínsecamente resilientes. Si una instancia de cómputo colapsa, el bucle de control detectará la ausencia de los servicios alojados en ella y los reprogramará de inmediato en otros nodos disponibles del clúster, sin requerir intervención humana (10).

Además, este enfoque facilita enormemente la escalabilidad y la gestión del ciclo de vida del software. Para escalar una aplicación ante un pico de tráfico, el administrador únicamente debe actualizar la declaración numérica del estado deseado, Kubernetes orquestará el despliegue de las nuevas instancias de forma progresiva. En conclusión, el paradigma declarativo permite tratar la infraestructura como código (IaC), garantizando que el entorno de producción sea predecible, auditable y altamente disponible (11).

2.4 Arquitectura Interna del Cluster

A nivel de infraestructura, un clúster de Kubernetes sigue un modelo de arquitectura distribuida maestro-esclavo, aunque en la terminología Cloud Native moderna se divide fundamentalmente en dos grandes dominios lógicos y físicos: el plano de control (Control Plane) y los nodos de trabajo (Worker Nodes).

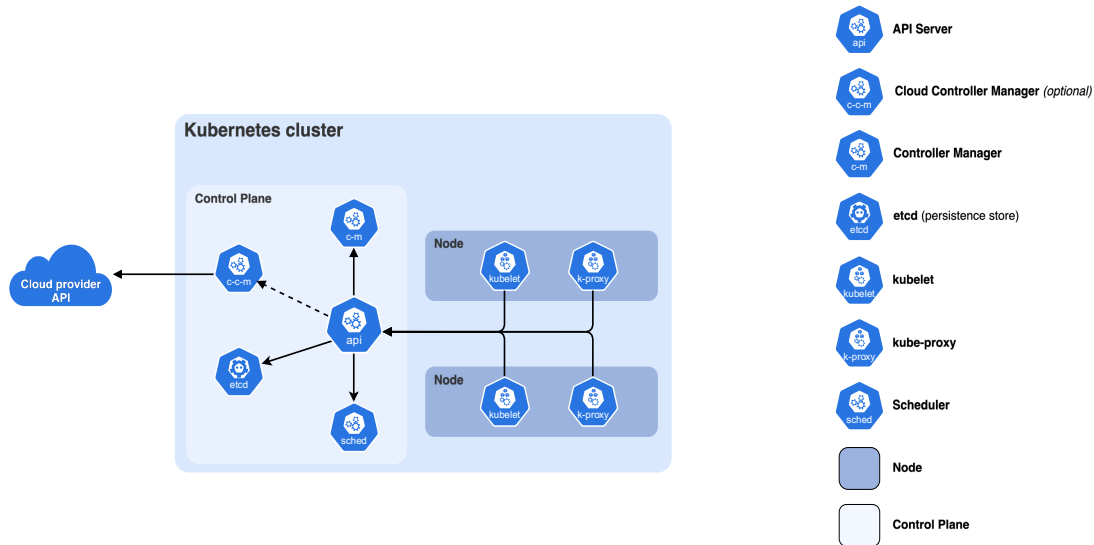


Figura 2-2 Arquitectura de un clúster de Kubernetes y componentes principales.

2.4.1 Plano de Control (Control Plane)

El plano de control actúa como el cerebro central del clúster. Su responsabilidad exclusiva es tomar decisiones globales sobre el sistema y detectar o responder a eventos (como arrancar un nuevo contenedor cuando falla otro). Para garantizar la alta disponibilidad en entornos de producción, estos componentes suelen ejecutarse de forma replicada en múltiples máquinas.

El Control Plane está compuesto por cinco elementos fundamentales (12):

- **kube-apiserver:** Es el componente frontal y el único que interactúa directamente con el exterior. Actúa como una API RESTful que recibe todas las peticiones, ya sean del administrador a través de la consola o de los propios nodos internos. Su función es autenticar, validar y procesar estas peticiones antes de actualizar el estado del clúster. Es la puerta de enlace de Kubernetes.
- **etcd:** Es una base de datos clave-valor distribuida, altamente disponible y consistente. Kubernetes la utiliza como su "única fuente de la verdad" (Single Source of Truth). Todo el estado del clúster, las configuraciones de red, las contraseñas y los manifiestos YAML que el administrador envía al API Server se almacenan aquí. Si se pierde la información de etcd, el clúster pierde la memoria de su estado.
- **kube-scheduler:** Es el componente encargado del despliegue dinámico. Su misión es vigilar si existen nuevos Pods (la unidad mínima de contenedores) creados pero aún no asignados a ninguna máquina física. Cuando detecta uno, evalúa los recursos requeridos (CPU, memoria), las restricciones de hardware y las políticas de afinidad, y selecciona el nodo de trabajo óptimo para ejecutarlo.

- **kube-controller-manager:** Es el motor lógico que ejecuta los bucles de reconciliación descritos en el paradigma declarativo. Agrupa múltiples controladores independientes en un solo proceso (por ejemplo, el Node Controller, que vigila si un nodo se cae, o el ReplicaSet Controller, que asegura que siempre haya el número exacto de copias de una aplicación en ejecución).
- **cloud-controller-manager:** Es el componente específico para despliegues en la nube. Enlaza el clúster con las API del proveedor Cloud (como AWS, GCP o Azure) para gestionar recursos externos. Permite, por ejemplo, que al definir un servicio de red, Kubernetes solicite automáticamente la creación de un balanceador de carga nativo del proveedor o gestione el almacenamiento persistente en discos de red en la nube (12).

2.4.2 Nodos de Trabajo (Worker Nodes)

Mientras que el plano de control se encarga de la orquestación global, los nodos de trabajo constituyen el entorno de ejecución de la plataforma. Son las instancias de cómputo (máquinas físicas o virtuales) las responsables de alojar y ejecutar las aplicaciones contenedorizadas que componen el sistema (13).

Para que una máquina pueda integrarse en el clúster y recibir órdenes del plano de control, debe contar con tres componentes fundamentales en ejecución continua:

- **kubelet:** Es el agente principal de cada nodo. Su función es registrar el nodo en el clúster y mantener una comunicación constante y bidireccional con el kube-apiserver. El kubelet recibe las especificaciones de los Pods (manifiestos YAML que indican qué contenedores deben ejecutarse) y se asegura de que esos contenedores se instancien y funcionen correctamente. Además, monitoriza el estado de salud del servidor (CPU, memoria y disco) y reporta dicha información al plano de control periódicamente. Si un contenedor falla, el kubelet es el encargado de intentar reiniciarlo localmente.
- **kube-proxy:** Es el componente encargado de la conectividad y de las reglas de red. Actúa como un proxy de red que se ejecuta en cada nodo e implementa el concepto de servicio de Kubernetes. Su misión es mantener las tablas de enrutamiento para garantizar que el tráfico de red, interno o externo, llegue de forma eficiente y balanceada al contenedor correcto, independientemente de en qué nodo físico se encuentre alojado.
- **Entorno de Ejecución de Contenedores (Container Runtime):** Aunque el kubelet da las órdenes, no ejecuta los contenedores por sí mismo. Para ello, cada nodo requiere un motor de ejecución compatible con el estándar CRI (Container Runtime Interface) de Kubernetes. Este componente es responsable de descargar las imágenes, desempaquetarlas e instanciar los procesos aislados. Actualmente, los entornos de ejecución más utilizados en la industria son containerd y CRI-O (13).

En conjunto, estos tres elementos transforman una simple máquina virtual en un nodo esclavo inteligente, capaz de abstraer su hardware subyacente y ofrecer sus recursos de computación a la flota de Kubernetes.

2.5 Entidades y Recursos de la API

2.5.1 Pods: La unidad mínima de despliegue

En el ecosistema de Kubernetes, la entidad atómica no es el contenedor. Un Pod representa la unidad de computación mínima, la más básica y desplegable, que se puede crear y gestionar dentro del clúster (14). Mediante esta capa de abstracción, Kubernetes se desacopla del motor de contenedores subyacente. Esto le permite estandarizar la ejecución de las cargas de trabajo, independientemente de si la infraestructura utiliza Docker, containerd o CRI-O.

Un Pod encapsula uno o más contenedores, recursos de almacenamiento compartido (volúmenes) y una única dirección IP lógica dentro de la red del clúster. Todos los contenedores que residen dentro de un mismo Pod comparten el mismo espacio de nombres de red (network namespace) y el mismo contexto de ejecución, lo que significa que pueden comunicarse entre sí utilizando localhost (14).

Aunque el modelo de despliegue más habitual es el de un contenedor por Pod (donde el Pod actúa simplemente como un envoltorio para un único microservicio), esta arquitectura permite la coexistencia de múltiples contenedores fuertemente acoplados entre sí. Esto da lugar a patrones de diseño avanzados, como el patrón Sidecar. En este escenario, un contenedor principal ejecuta la lógica de negocio de la aplicación, mientras que un contenedor secundario (el sidecar) se ejecuta en paralelo para gestionar tareas auxiliares y transversales, como la recopilación de logs, la sincronización de archivos o la gestión de certificados de seguridad perimetral (15).

Por último, a nivel operativo, es fundamental comprender la naturaleza efímera de los Pods. Estas entidades no están diseñadas para ser duraderas ni para mantener un estado persistente. Si el nodo en el que se ejecuta sufre un fallo de hardware, o si el proceso interno colapsa, Kubernetes no intenta resucitar esa instancia específica. En su lugar, el sistema la descarta y el Control Plane programa la creación de un Pod completamente nuevo para sustituirlo (14). Esta volatilidad inherente hace indispensable el uso de entidades de orden superior que gestionen su ciclo de vida y aseguren la alta disponibilidad de la aplicación.

Para ilustrar este concepto de forma práctica, la especificación declarativa de un Pod básico que ejecuta un servidor web, escrita en formato YAML, se estructuraría de la siguiente manera:

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
  labels:
    app: web
spec:
  containers:
    - name: web-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

En este manifiesto se observa cómo se define el estado deseado de la unidad mínima, indicando la imagen del contenedor subyacente y el puerto interno que expone, aislando así la configuración de la infraestructura física.

2.5.2 Controladores: Deployments y ReplicaSets

Como se ha establecido en el apartado anterior, los Pods son entidades por naturaleza efímeras. Para construir un sistema resiliente y de alta disponibilidad, es necesario delegar la gestión de su ciclo de vida en entidades de orden superior denominadas controladores. En la arquitectura de Kubernetes, los dos controladores fundamentales para la ejecución de aplicaciones sin estado (Stateless) son los ReplicaSets y los Deployments.

- **ReplicaSets: Garantizando la disponibilidad**

El propósito fundamental de un ReplicaSet es mantener un conjunto estable de réplicas de Pods idénticos en ejecución en todo momento (16). Operando bajo el paradigma declarativo, el administrador define el número exacto de instancias deseadas. El ReplicaSet utiliza selectores de etiquetas para identificar y monitorizar los Pods bajo su control. A través del bucle de reconciliación, si un nodo del clúster colapsa y, en consecuencia, un Pod desaparece, este controlador solicita de inmediato al plano de control la creación de un nuevo Pod para restaurar la cuota definida, garantizando así la tolerancia a fallos.

- **Deployments: Gestión del ciclo de vida y actualizaciones**

Aunque los ReplicaSets garantizan la alta disponibilidad estática, carecen de mecanismos integrados para gestionar la evolución del software. Para solventar esta limitación, se utiliza el Deployment, una abstracción de nivel superior que envuelve y administra de forma automatizada los ReplicaSets (17).

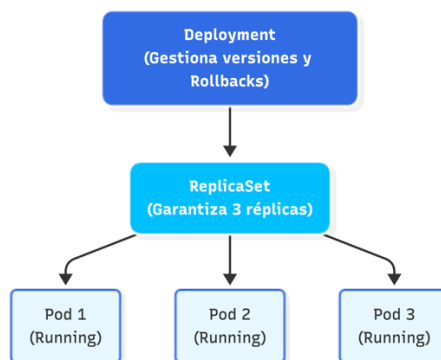


Figura 2-3 Jerarquía de controladores lógicos: El Deployment administra el ReplicaSet, que a su vez garantiza la ejecución continua de los Pods.

En la práctica industrial, los ingenieros de plataforma interactúan exclusivamente con los Deployments. La principal ventaja de esta entidad es su capacidad para orquestar transiciones de estado de forma segura y predecible. Al solicitar una actualización de la imagen de un contenedor, el Deployment no detiene abruptamente el servicio. En su lugar, ejecuta una actualización progresiva (Rolling Update): crea un nuevo ReplicaSet con la nueva versión y lo escala gradualmente, al mismo tiempo que reduce paulatinamente las réplicas del ReplicaSet obsoleto. Si se detecta inestabilidad o un alto índice de errores durante el proceso, el Deployment permite ejecutar una reversión inmediata (rollback) al estado anterior, garantizando despliegues sin interrupción del servicio (Zero Downtime Deployments) (17).

Para comprender cómo se plasman estas capacidades de abstracción y escalado de forma declarativa, la especificación canónica de un objeto Deployment parametrizado para mantener tres réplicas concurrentes se estructura a través de un archivo YAML de la siguiente manera:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web-container
          image: nginx:latest
```

En este manifiesto teórico se evidencian los tres pilares de un controlador: el número de instancias deseadas (replicas: 3), el mecanismo de emparejamiento por etiquetas (matchLabels) que el ReplicaSet interno utilizará en el bucle de reconciliación, y la plantilla inmutable (template) que define la configuración técnica del contenedor a desplegar.

2.5.3 Redes y Exposición: Services e Ingress

Una vez que los Deployments garantizan la ejecución continua y escalable de los microservicios, surge un nuevo desafío arquitectónico: la conectividad y la localización dinámica de los servicios. Como se ha expuesto en apartados anteriores, los Pods son efímeros. Al ser destruidos y reemplazados por el bucle de reconciliación, adquieren direcciones IP nuevas de forma dinámica. Si un microservicio frontend necesita consumir datos de un microservicio backend, resulta operativamente inviable depender de direcciones IP volátiles.

Para resolver esta problemática de enrutamiento, Kubernetes introduce la entidad **Service**. Un Service es una abstracción lógica que agrupa un conjunto dinámico de Pods y les asigna una dirección IP virtual, estable y permanente, así como un registro de resolución de nombres (DNS) interno (18). Además de proveer una identidad estática de red, el Service actúa como un balanceador de carga de capa 4 (TCP/UDP), distribuyendo el tráfico entrante de forma equitativa y transparente entre todas las réplicas sanas que respaldan la aplicación.

En despliegues sobre proveedores de cloud, el Service puede configurarse como tipo LoadBalancer. Esta configuración invoca al cloud-controller-manager para aprovisionar automáticamente un balanceador de carga físico en la infraestructura de la nube, lo que permite la entrada de tráfico desde Internet.

Sin embargo, a medida que la topología de la aplicación crece, asignar un LoadBalancer público a cada microservicio resulta una práctica ineficiente. Para solucionar esto y optimizar la exposición pública, se utiliza la entidad Ingress.

A diferencia de un Service, un Ingress opera en la capa 7 del modelo OSI (protocolos HTTP/HTTPS). Actúa como un enrutador inteligente en el borde del clúster, permitiendo consolidar el tráfico externo a través de un único punto de entrada público (19). Mediante la definición de reglas declarativas, el Ingress evalúa la petición HTTP entrante y la enruta al Service interno correspondiente según la ruta de la URL (por ejemplo, derivando el tráfico de midominio.com/api al backend y de midominio.com/app al frontend). Además, esta entidad centraliza tareas de seguridad perimetral, como la terminación de certificados criptográficos (SSL/TLS), lo que descarga de la responsabilidad computacional a los contenedores individuales. El flujo completo de este enrutamiento se puede observar en la **Figura 2-4**.

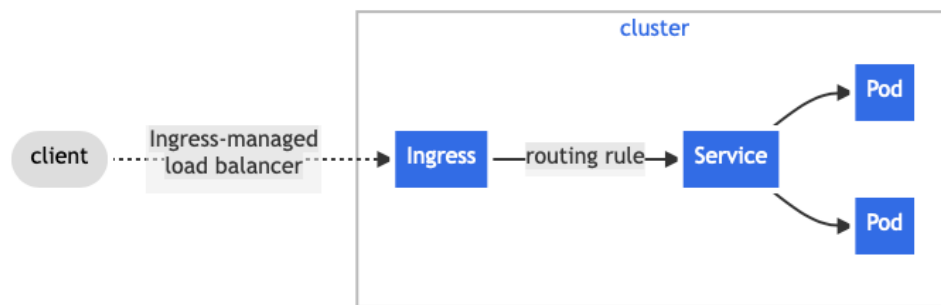


Figura 2-4 Flujo de tráfico de red en Kubernetes: enrutamiento externo mediante Ingress al Service y balanceo de carga final hacia los Pods.

Para materializar esta infraestructura de red de manera complementaria bajo el paradigma declarativo, se definen los recursos correspondientes en archivos de configuración. A continuación, se muestra en primer lugar la especificación teórica de un servicio interno aislado (ClusterIP) y, en segundo lugar, el manifiesto del *Ingress* encargado de interceptar el tráfico exterior y redirigirlo hacia dicho servicio:

```

apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

```

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
spec:
  rules:
    - host: midominio.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: web-service
                port:
                  number: 80

```

En este conjunto de manifiestos se evidencia la cohesión del modelo de red de Kubernetes: el objeto *Ingress* expone el dominio público *midominio.com* y evalúa el prefijo */api* para redirigir la petición web al recurso interno denominado *web-service* a través del puerto 80. Finalmente, el *Service* intercepta dicha solicitud y, utilizando su selector dinámico (*app: web*), distribuye el flujo de datos a los puertos internos (*targetPort: 80*) de los contenedores en estado saludable.

2.5.4 Almacenamiento y Estado: Persistent Volumes

Como se analizó en la definición de los Pods, el sistema de archivos nativo de un contenedor es efímero. Si un contenedor falla y es reiniciado por el kubelet, o si un Pod es reprogramado en otro nodo, todos los datos generados y almacenados en su disco local se pierden de forma irrecuperable. Aunque este comportamiento es ideal para microservicios sin estado (Stateless), resulta inviable al desplegar aplicaciones con estado (Stateful), como bases de datos, repositorios de archivos o colas de mensajería.

Para resolver este problema y desacoplar el ciclo de vida del almacenamiento del ciclo de vida computacional del Pod, la API de Kubernetes implementa un subsistema de almacenamiento persistente basado en dos recursos fundamentales (20):

- **Persistent Volume (PV):** Es una porción de almacenamiento físico en la infraestructura que ha sido aprovisionada para el clúster. Es un recurso global y su ciclo de vida es completamente independiente de cualquier Pod individual.
- **Persistent Volume Claim (PVC):** Es una declaración de intenciones o solicitud de almacenamiento realizada por una aplicación. En lugar de lidiar con los detalles técnicos del disco subyacente, el ingeniero define en el PVC sus requerimientos (por ejemplo, "necesito 10 GB de almacenamiento con permisos de lectura y escritura"). El plano de control se encarga de buscar un PV disponible que cumpla con esta demanda y de enlazarlo automáticamente.

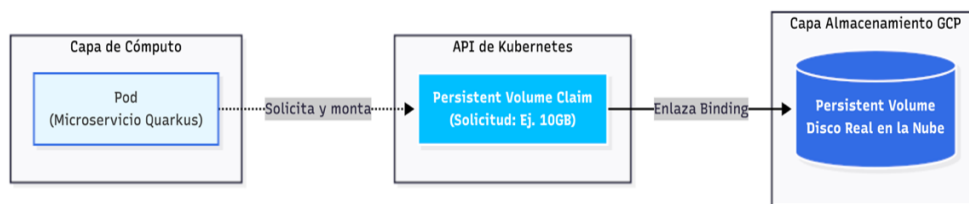


Figura 2-5 Desacoplamiento del almacenamiento en Kubernetes mediante PV y PVC.

En despliegues modernos en proveedores de cloud (como AWS, GCP o Azure), la creación manual de Persistent Volumes resulta ineficiente a gran escala. Para solucionar esto, Kubernetes introduce el concepto de StorageClass (Clase de Almacenamiento). Esta entidad habilita el aprovisionamiento dinámico de discos (21).

Al configurar una StorageClass, cuando una aplicación emite un PVC, el clúster no requiere que el disco ya exista. En su lugar, el cloud-controller-manager intercepta la petición, se comunica directamente con la API del proveedor en la nube y ordena la creación de un nuevo disco duro virtual sobre la marcha, conectándolo y montándolo de forma transparente en el nodo donde reside el Pod.

Para ilustrar este mecanismo de desacoplamiento de forma declarativa, a continuación se muestra la especificación de un PersistentVolumeClaim que solicita 10 GB de almacenamiento, seguida de la configuración teórica de un contenedor de base de datos que monta dicho volumen:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: database-pvc
```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: database-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard
```

En este ejemplo se ve la separación de responsabilidades: la aplicación simplemente solicita la capacidad de disco que necesita mediante el PVC (database-pvc), delegando en la clase de almacenamiento (standard) el aprovisionamiento físico en la nube. Posteriormente, el Pod asocia esa reclamación a nivel global (volumes) y la inyecta en la ruta interna específica donde el motor de base de datos escribe su información (volumeMounts), garantizando la persistencia de los datos frente a la destrucción del contenedor.

2.5.5 Configuración y Seguridad

Para finalizar la definición de las entidades fundamentales de Kubernetes, es necesario abordar la gestión de las variables de entorno y de las credenciales. Una de las mejores prácticas en el desarrollo de aplicaciones nativas en la nube, basada en la metodología Twelve-Factor App, es la separación entre el código fuente y su configuración. Incrustar cadenas de conexión, credenciales o variables de entorno directamente en la imagen del contenedor compromete tanto la seguridad como la portabilidad del artefacto (ya que obligaría a compilar una imagen distinta para el entorno de desarrollo y otra para producción).

Para implementar esta separación, Kubernetes proporciona dos recursos específicos que inyectan la configuración en los Pods en tiempo de ejecución:

- **ConfigMap:** Es un objeto de la API diseñado para almacenar datos no confidenciales en formato clave-valor. Permite desacoplar la configuración específica del entorno de las imágenes de los contenedores (22). Un ConfigMap puede contener desde simples variables de entorno (como la URL de un servicio externo o el nivel de logs deseado) hasta archivos de configuración completos (como el archivo `nginx.conf` de un servidor web). Cuando el kubelet instancia un Pod, lee estos datos e inyecta los datos en el contenedor, ya sea como variables de entorno nativas o como archivos de solo lectura en su sistema de archivos.
- **Secret:** Es una entidad operativamente idéntica al ConfigMap, pero diseñada exclusivamente para contener una pequeña cantidad de datos altamente confidenciales, tales como contraseñas de bases de datos, tokens de acceso a APIs o certificados criptográficos (TLS) (23). La ventaja arquitectónica de utilizar Secrets es que los desarrolladores no necesitan incluir esta información crítica en el código de la aplicación ni en los manifiestos YAML que se suben a los repositorios de control de versiones. Además, Kubernetes implementa medidas adicionales con los Secrets, como enviarlos únicamente a los nodos en los que se ejecuta el Pod que los requiere y

mantenerlos en memoria RAM en lugar de escribirlos en el almacenamiento persistente del nodo.

Para ilustrar esta separación de configuración, el siguiente manifiesto muestra cómo un contenedor puede inyectar a la vez un dato público desde un ConfigMap y una credencial de forma segura desde un Secret, mapeándolos como variables de entorno (API_URL y DATABASE_PASSWORD) sin exponer sus valores reales en el manifiesto principal:

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app-container
      image: mi-aplicacion:v1
      env:
        # Inyección de configuración no confidencial
        - name: API_URL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: api_url
        # Inyección segura de credenciales
        - name: DATABASE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: eventos-db-secret
              key: POSTGRES_PASSWORD
```

2.6 Despliegue en la Nube: Iaas frente a CaaS

Como hemos visto a lo largo de este capítulo, Kubernetes es una plataforma increíblemente potente, pero operar su arquitectura interna no es una tarea sencilla. Mantener un clúster en producción implica asegurar que el plano de control nunca se caiga, gestionar los certificados de seguridad, realizar copias de seguridad de la base de datos de etcd y aplicar actualizaciones sin interrumpir el servicio a los usuarios. Ante este reto de mantenimiento, la industria nos ofrece dos caminos principales para llevar Kubernetes a la nube (24).

2.6.1 Despliegue manual: Infraestructura como Servicio (IaaS)

La primera opción es alquilar servidores vacíos en la nube (máquinas virtuales puras) y que el equipo de ingeniería asuma la responsabilidad de instalar y configurar cada componente de Kubernetes desde cero. Esto es lo que en la industria se conoce popularmente como "Kubernetes the hard way". Aunque este enfoque otorga un control absoluto sobre el sistema, el coste en tiempo y esfuerzo es altísimo. Obliga a los desarrolladores a dedicar la mayor parte de su jornada a tareas de mantenimiento puro, lo que les resta tiempo para programar la lógica de negocio real.

2.6.2 Delegar la complejidad: Contenedores como Servicio (CaaS)

Para solucionar este desgaste operativo, proveedores líderes como Amazon (con EKS), Google (GKE) o Microsoft (AKS) ofrecen Kubernetes como un servicio gestionado o CaaS (25). En este modelo, el proveedor Cloud asume toda la responsabilidad del plano de control del clúster (Control Plane). Ellos se encargan de que el API Server, etcd y los distintos

controladores estén siempre disponibles, actualizados y seguros frente a fallos. El ingeniero de plataforma, por su parte, solo tiene que preocuparse por gestionar los nodos de trabajo (Worker Nodes) y desplegar sus microservicios. En definitiva, se alquila un orquestador ya montado, blindado y listo para recibir órdenes.

2.6.3 Conclusión arquitectónica y decisión del proyecto

En el desarrollo Cloud Native moderno, montar Kubernetes desde cero rara vez compensa el esfuerzo, a menos que la organización tenga necesidades de personalización de infraestructura extremas. Confiar en soluciones CaaS se ha convertido en el estándar de la industria porque reduce los errores humanos y permite a los equipos centrarse en aportar valor real mediante el desarrollo de software. Es precisamente por esta eficiencia operativa, seguridad y ahorro de tiempo que el despliegue práctico de este Trabajo de Fin de Grado se realizará en un servicio de Kubernetes gestionado en la nube.

2.7 Análisis Comparativo (Kubernetes vs Docker Swarm)

2.7.1 Filosofía de Diseño y Arquitectura Subyacente

Para comprender las diferencias operativas entre los principales orquestadores, es necesario entender la visión con la que fueron diseñados. Las herramientas tecnológicas no son neutrales, su arquitectura dicta cómo los ingenieros deben interactuar con ellas. En este sentido, Docker Swarm y Kubernetes parten de visiones distintas sobre la gestión de contenedores.

2.7.1.1 Docker Swarm: La filosofía de la simplicidad nativa

Docker Swarm nació con el objetivo de ser la extensión lógica del ecosistema Docker, priorizando la facilidad de uso y la integración directa. Su filosofía principal es ofrecer la menor complicación posible al desarrollador. Si un ingeniero ya conoce el funcionamiento básico de Docker, el salto a la orquestación con Swarm resulta casi transparente.

Arquitectónicamente, Swarm apuesta por un modelo "todo en uno". No requiere instalar software adicional, ya que el motor de orquestación está integrado en el propio Daemon de Docker. Utiliza un modelo de consenso basado en el algoritmo Raft para coordinar los nodos manager de forma ligera (26). Es una herramienta diseñada para desplegarse rápidamente y perfecta para equipos que buscan resultados inmediatos sin incurrir en una alta complejidad operativa.

2.7.1.2 Kubernetes: La filosofía de la modularidad y el estado deseado

Por el contrario, Kubernetes no fue diseñado para ser sencillo de instalar, sino para ser infalible a escala masiva. Su filosofía central no gira en torno a los contenedores de forma aislada, sino al paradigma declarativo y a la abstracción total de la infraestructura mediante los objetos de su API (27).

A nivel de arquitectura, Kubernetes es un sistema profundamente desacoplado y modular. No es un programa único, sino una colección de componentes independientes (como el API Server o el Scheduler) que colaboran entre sí. Esta modularidad permite que el sistema sea extensible. Se asume que las infraestructuras grandes son propensas a fallos, por lo que su diseño delega la responsabilidad de mantener el orden en sus bucles de reconciliación.

2.7.1.3 El contraste arquitectónico

En resumen, la diferencia en la filosofía es clara: mientras que Docker Swarm prioriza la experiencia del desarrollador y la inmediatez, ocultando la complejidad bajo una capa sencilla, Kubernetes asume una curva de aprendizaje inicial a cambio de ofrecer un sistema modular, resiliente y preparado para sostener ecosistemas empresariales críticos.

2.7.2 Gestión del Clúster, Escalabilidad y Rendimiento

Una vez comprendidas sus bases filosóficas, es necesario analizar cómo se comportan ambas herramientas en la realidad operativa. No es lo mismo gestionar un clúster de tres servidores que uno de trescientos, y aquí es donde las diferencias de diseño se traducen en ventajas o inconvenientes operativos.

2.7.2.1 Gestión y puesta en marcha: Simplicidad frente al control

En el día a día de un ingeniero, la gestión del clúster con Docker Swarm resulta bastante sencilla. Al estar integrado en el motor de Docker, convertir un grupo de máquinas en un clúster es tan simple como ejecutar un único comando: `docker swarm init`. No hay bases de datos externas que configurar ni certificados que generar manualmente. La herramienta se encarga de todo internamente y de forma transparente (28).

Por el contrario, la gestión de un clúster de Kubernetes es conocida por su complejidad. Incluso con herramientas que simplifican el proceso, el administrador debe lidiar con la configuración de múltiples componentes y con una red interna mucho más compleja. Sin embargo, esta dificultad inicial se traduce en un control total. Kubernetes permite definir políticas de seguridad y límites de recursos con una precisión que Swarm no puede alcanzar.

2.7.2.2 Escalabilidad: ¿Qué tan rápido podemos crecer?

Cuando hablamos de escalabilidad, ambos orquestadores pueden manejar miles de contenedores, pero lo hacen de distintas maneras. Docker Swarm destaca por su velocidad de reacción, al ser un sistema más ligero, es capaz de arrancar cientos de réplicas de un contenedor en poco tiempo.

Sin embargo, Kubernetes es el rey de la escalabilidad inteligente. Mientras que en Swarm la escalación suele ser una orden manual del administrador, Kubernetes introduce el Horizontal Pod Autoscaler (HPA). Este componente monitoriza constantemente el uso de CPU o memoria y, si detecta un pico de tráfico, toma la decisión autónoma de crear más réplicas de la aplicación sin que ningún humano tenga que intervenir (29). Una vez que el tráfico disminuye, el sistema destruye las copias sobrantes para reducir costes.

2.7.2.3 Rendimiento y eficiencia

En términos de rendimiento puro, Docker Swarm suele ganar en escenarios pequeños porque consume menos recursos del sistema para mantenerse en funcionamiento. Sin embargo, en infraestructuras masivas, el rendimiento de Kubernetes es más previsible y estable, ya que su avanzado Scheduler sabe repartir la carga de trabajo de forma mucho más eficiente entre los servidores disponibles, evitando que unos trabajen demasiado mientras otros están poco utilizados.

2.7.3 Tolerancia a Fallos y Alta Disponibilidad

En cualquier despliegue en la nube, asumir que la infraestructura física es infalible es un grave error de diseño. Los servidores colapsan y las redes sufren interrupciones. La diferencia entre una caída total del sistema y un simple contratiempo imperceptible para el usuario radica en cómo el orquestador reacciona ante estos desastres.

2.7.3.1 Docker Swarm: Recuperación reactiva y consenso

Docker Swarm ofrece una alta disponibilidad robusta y muy directa. Su cerebro (los nodos *mánager*) utiliza el algoritmo Raft para mantener una copia exacta del estado del clúster. Si un *mánager* principal se cae, los demás votan en milisegundos para elegir a un nuevo líder, evitando que el clúster quede descabezado.

A nivel de ejecución, si una máquina de trabajo (*worker node*) se apaga repentinamente, Swarm detecta la pérdida de conexión y reasigna los contenedores caídos a otras máquinas sanas (30). Sin embargo, sus mecanismos de comprobación son algo limitados. Por defecto, Swarm asume que si el proceso del contenedor sigue ejecutándose (es decir, no se ha cerrado), todo va bien. Esto puede ser un problema si la aplicación se congela internamente sin cerrarse.

2.7.3.2 Kubernetes: Autocuración (Self-healing) microscópica y macroscópica

Aquí es donde Kubernetes demuestra por qué es el estándar industrial para sistemas críticos. Su enfoque no se limita a verificar si un servidor está encendido o apagado, sino que adopta un paradigma de *self-healing* en todos los niveles.

En lugar de confiar en que un proceso activo indique que la aplicación funciona, Kubernetes obliga a los desarrolladores a configurar sondas de salud avanzadas, conocidas como *Liveness Probes* y *Readiness Probes* (31). Estas sondas interrogan continuamente al código (por ejemplo, mediante una petición HTTP a una ruta de diagnóstico). Si la aplicación se bloquea y deja de responder correctamente, Kubernetes mata quirúrgicamente ese contenedor específico y levanta una copia limpia, todo ello de forma autónoma. A un nivel superior, si un centro de datos entero desaparece de la red, los controladores de Kubernetes migrarán masivamente la carga de trabajo a otros nodos sanos, garantizando que el usuario final no perciba ninguna interrupción del servicio.

2.7.4 Gestión de Redes y Almacenamiento Persistente

En cualquier sistema distribuido, la forma en que los componentes se comunican y guardan sus datos determina la robustez de la arquitectura. Mientras que Docker Swarm busca que la red y el disco simplemente funcionen sin que el usuario perciba la complejidad, Kubernetes prefiere exponer esa complejidad para ofrecer una flexibilidad total y cumplir con los estándares industriales.

2.7.4.1 Redes: Simplicidad frente a estándares de arquitectura

La red de Docker Swarm es muy sencilla de gestionar. Utiliza redes de tipo *overlay* que permiten que los contenedores en distintos servidores se comuniquen entre sí como si estuvieran conectados al mismo cable. Además, incluye por defecto el *Routing Mesh*, un mecanismo transparente que redirige automáticamente cualquier petición que llegue al clúster al contenedor correcto, independientemente de en qué nodo físico se encuentre (32). Es una gran solución para proyectos medianos, pero se vuelve difícil de gestionar cuando necesitamos reglas de seguridad muy estrictas o redes muy segmentadas.

Kubernetes, por el contrario, no impone una red única, sino que define un estándar llamado CNI (*Container Network Interface*). Esto permite al ingeniero elegir el motor de red que mejor se adapte a sus necesidades. En Kubernetes, cada *Pod* recibe su propia dirección IP única y real dentro del clúster, lo que elimina la necesidad de mapear puertos y facilita la comunicación entre microservicios (33). Aunque es más complejo de configurar inicialmente, permite aplicar políticas de red que actúan como un cortafuegos quirúrgico entre aplicaciones, algo que Swarm no puede lograr de forma nativa.

2.7.4.2 Almacenamiento: Persistencia local frente a discos inteligentes

El almacenamiento es el punto en el que los orquestadores más sufren. En Docker Swarm, la gestión de datos suele depender de volúmenes locales o de montar carpetas de red (NFS) de forma manual (34). El problema surge cuando un servidor falla y el contenedor debe moverse a otro, si el disco no está físicamente allí o el montaje de red falla, los datos no estarán disponibles.

Aquí es donde Kubernetes marca la diferencia técnica. Gracias al estándar CSI (Container Storage Interface), Kubernetes puede comunicarse directamente con la infraestructura de tu proveedor de nube. Como se analizó en un apartado anterior, el sistema de Persistent Volumes permite que, si una base de datos necesita moverse de un servidor a otro, el orquestador se encargue automáticamente de desenchufar el disco virtual en un sitio y enchufarlo en el nuevo destino sin intervención humana (35). Esta capacidad de gestionar el almacenamiento de forma dinámica es lo que permite que las aplicaciones en Kubernetes sean verdaderamente resistentes a fallos de hardware.

2.7.5 Ecosistema, Extensibilidad y Curva de Aprendizaje

Más allá de los datos técnicos de red o de CPU, el éxito de una tecnología en el mundo real depende de su comunidad y de lo fácil que sea para un equipo de ingeniería adoptarla. En este aspecto, Docker Swarm y Kubernetes representan los dos extremos del espectro: la inmediatez frente a la especialización.

2.7.5.1 Curva de aprendizaje: El “muro” de Kubernetes

Si algo define a Docker Swarm, es su baja barrera de entrada. Como se ha mencionado, su integración con la CLI de Docker permite que un desarrollador sea productivo en cuestión de horas. No hay conceptos nuevos que aprender, un Service en Swarm es casi idéntico a lo que un programador ya hace en su máquina local. Es la opción ideal para proyectos en los que el tiempo es oro y no se dispone de un equipo dedicado exclusivamente a sistemas.

Kubernetes, en cambio, presenta lo que muchos ingenieros denominan una curva de aprendizaje en forma de muro. Dominar Kubernetes no solo implica aprender a gestionar contenedores, sino también entender conceptos de red, seguridad y almacenamiento específicos de su API (36). Requiere una inversión de tiempo significativa y, a menudo, la presencia de un rol especializado (DevOps o SRE). Sin embargo, este esfuerzo se ve recompensado con una capacidad de control que Swarm, por su sencillez, no puede ofrecer.

2.7.5.2 Ecosistema y Extensibilidad: El gigante de la CNCF

En el ámbito del ecosistema, Kubernetes se posiciona claramente como la opción dominante. Al ser el proyecto estrella de la Cloud Native Computing Foundation (CNCF), cuenta con el respaldo de gigantes como Google, Amazon y Microsoft. Esto se traduce en un gran mercado de herramientas. Si necesitas monitorización, tienes Prometheus. Si necesitas seguridad de red, tienes Istio. Si necesitas la gestión de paquetes, tienes Helm (36).

Además, Kubernetes ofrece una extensibilidad ilimitada. Mediante los llamados Custom Resource Definitions (CRDs) y los Operators, puedes enseñar a Kubernetes a gestionar cosas que no vienen de fábrica, como bases de datos complejas o certificados SSL de forma automática (37). Docker Swarm es un sistema más cerrado, lo que ves es lo que hay. Es excelente en lo que hace, pero si tus necesidades se salen de lo estándar, Swarm ofrece muy pocos mecanismos para ampliar sus funciones de forma nativa.

Tabla 2-1 Resumen comparativo de características operativas entre Docker Swarm y Kubernetes.

Característica	Docker Swarm	Kubernetes
Filosofía de diseño	Simplicidad nativa e inmediatez.	Modularidad, abstracción y estado deseado.
Curva de aprendizaje	Baja (integrado en la CLI de Docker).	Alta (curva en forma de muro)
Escalabilidad	Reactiva y manual	Inteligente y autónoma (vía HPA)
Tolerancia a fallos	Básica (verificación de proceso activo).	Avanzada (Self-healing y Liveness/Readiness Probes)
Redes y almacenamiento	Soluciones nativas integradas (Overlay). Volúmenes principalmente locales	Estandares modulares (CNI y CSI). Aprovisionamiento dinámico en la nube.
Ecosistema y Extensibilidad	Cerrado. Funcionalidades limitadas al estándar.	Masivo (Respaldado por la CNCF). Extensible (CRDs, Operators).

2.7.6 Conclusión: Justificación Tecnológica de la Elección

Tras el análisis detallado realizado en los apartados anteriores, queda claro que no existe una herramienta mejor en términos absolutos, sino una más adecuada para cada escenario. La decisión entre Docker Swarm y Kubernetes es una decisión entre simplicidad operativa y potencia y escalabilidad profesionales.

2.7.6.1 ¿Por qué no Docker Swarm?

A pesar de ser una herramienta brillante por su agilidad y su curva de aprendizaje nula, Docker Swarm tiene limitaciones que no son compatibles con los objetivos del proyecto. Su enfoque en la simplicidad se queda corto cuando necesitamos gestionar el almacenamiento de forma dinámica en la nube o requerimos políticas de red más granulares. Aunque es ideal para prototipos rápidos o pequeñas infraestructuras internas (38), para un sistema que aspire a seguir las mejores prácticas de la industria Cloud Native, Swarm se percibe hoy en día como una solución demasiado rígida.

2.7.6.2 Justificación de la elección de Kubernetes

Para el desarrollo de este Trabajo de Fin de Grado, se ha seleccionado Kubernetes como el orquestador de referencia por los siguientes motivos estratégicos:

1. **Estándar de la industria:** Kubernetes no es solo una herramienta, es el ecosistema donde ocurre toda la innovación en la nube actualmente. Aprender y desplegar sobre Kubernetes garantiza que el proyecto sea tecnológicamente relevante y alineado con lo que demandan las empresas punteras (39).
2. **Resiliencia total:** La capacidad de autocuración (self-healing) y las sondas de salud avanzadas aseguran que la aplicación pueda recuperarse por sí sola ante fallos de servidores, lo cual es vital para garantizar la alta disponibilidad.

3. **Abstracción de la Nube:** Al usar servicios gestionados (CaaS), podemos aprovechar toda la potencia de Kubernetes sin la complejidad de montar el clúster a mano, logrando un equilibrio perfecto entre control y eficiencia operativa.
4. **Futuro y Extensibilidad:** Gracias a su ecosistema masivo, el proyecto está preparado para crecer. Cualquier funcionalidad que necesitemos añadir en el futuro ya existe y está probada por la comunidad.

En definitiva, se opta por Kubernetes no por ser el camino más fácil, sino por ser el más robusto, profesional y escalable. Esta elección sienta las bases para los siguientes capítulos, en los que pasaremos del diseño teórico a la implementación real de la arquitectura en un entorno de nube pública, como Google Cloud Platform.

3. Especificación Requisitos

3.1 Visión General del Sistema

Este TFG tiene como objetivo principal el diseño y desarrollo de una plataforma digital orientada a la gestión de eventos y a la reserva automatizada de entradas. El sistema está diseñado para atender tanto a los administradores de la oferta de ocio como a los usuarios finales que desean reservar en dichos eventos.

Para garantizar un funcionamiento eficiente y modular, las funcionalidades del sistema se han dividido en dos grandes bloques o microservicios:

- **Microservicio de Eventos:** Encargado de gestionar el catálogo. Centraliza toda la información de la oferta de ocio, incluidos recintos, aforos y horarios, y es el responsable absoluto de mantener actualizado el inventario de entradas.
- **Microservicio de Reservas:** Actúa como el motor transaccional. Es el responsable de atender a los usuarios, procesar sus intenciones de compra y comunicarse con el catálogo para verificar que hay aforo disponible antes de confirmar cualquier compra.

El sistema, en su conjunto, debe operar de forma resiliente, asegurando en todo momento la integridad de las ventas, evitando la sobreventa ante alta concurrencia y garantizando que el usuario reciba un servicio fluido e ininterrumpido.

3.2 Requisitos Funcionales

Los requisitos funcionales describen lo que el sistema debe hacer para aportar valor al negocio y a los usuarios. Para simplificar, he dividido estas funcionalidades según el microservicio que se encarga de cada una, añadiendo un apartado para las reglas importantes que afectan a todo el sistema.

3.2.1 Módulo de Catálogo de Eventos (Microservicio ws-catalogo)

Este módulo es la parte visual del sistema. Su objetivo es ofrecer información sobre los eventos y controlar el inventario de entradas.

- **Consulta de la cartelera:** Cualquier usuario debe poder ver la lista completa de eventos activos y consultar rápidamente datos clave como el nombre, la disciplina, la fecha y si aún quedan plazas.
- **Detalle del evento:** El sistema permitirá acceder a la ficha específica de un evento para consultar su información en profundidad, incluida la descripción completa y el recinto donde se celebra.

- **Actualización de aforo en vivo:** Cada vez que se cierre una venta, este módulo restará instantáneamente las entradas compradas al aforo total, garantizando que el stock mostrado sea siempre el real.
- **Cierre automático por aforo completo:** El sistema vigilará el stock en todo momento. En el mismo momento en que las entradas de un evento lleguen a cero, su estado cambiará automáticamente a LLENO, lo que bloqueará cualquier intento futuro de compra sin necesidad de intervención de un administrador.

3.2.2 Módulo de Reservas (Microservicio ws-reservas)

Este módulo es el motor de ventas. Su trabajo consiste en procesar de forma segura las intenciones de compra de los usuarios.

- **Creación de la reserva:** El sistema permitirá a los usuarios solicitar un número determinado de entradas para un evento, registrando quién realiza la compra y calculando el precio final.
- **Gestión segura del ciclo de vida:** Para evitar errores, ninguna compra se aprueba directamente. Toda reserva nace primero en estado PENDIENTE. Solo cuando el sistema confirma internamente que hay sitio libre en el evento, la reserva pasa a CONFIRMADA. Si algo falla, se marca como CANCELADA.

3.2.3 Reglas de Negocio

Vender entradas por internet implica enfrentarse a picos de tráfico y a posibles fallos de conexión. El sistema debe garantizar que los datos nunca se corrompan.

- **Blindaje contra falta de stock:** El sistema rechazará cualquier solicitud de compra que intente adquirir más entradas de las disponibles en el recinto.
- **Prevención de Sobreventa:** El sistema está preparado para soportar compras masivas simultáneas. Si dos o más usuarios intentan comprar la última entrada libre al mismo tiempo, el sistema dará prioridad a uno y rechazará a los demás, asegurando que la misma entrada no se venda dos veces.
- **Tolerancia a cortes de red:** Si la comunicación interna entre las reservas y el catálogo se interrumpe temporalmente durante una compra, el sistema debe anular la transacción. De este modo, nos aseguramos de que no queden entradas sin confirmación.

3.3 Diseño y Especificación de la API REST

Dado que el sistema se ha diseñado con una arquitectura distribuida, sus distintos módulos necesitan comunicarse entre sí y atender las peticiones de usuarios externos. Para ello, el sistema expone un conjunto de interfaces basadas en el modelo REST.

Los principales puntos de acceso (endpoints) del sistema serán:

3.3.1 Endpoints del Catálogo de Eventos

Permiten leer los eventos disponibles y actualizar el stock.

- **Listado General de Eventos (GET /eventos)**

- **Propósito:** Actúa como escaparate. Cualquier cliente puede llamar a esta ruta para obtener el catálogo completo.
- **Intercambio de datos:** No requiere parámetros de entrada. El sistema devuelve una lista en formato de texto (JSON) con la información pública de todos los eventos activos.
- **Detalle de un Evento (GET /eventos/{id})**
 - **Propósito:** Permite consultar la información ampliada de un evento específico.
 - **Intercambio de datos:** Recibe el identificador único del evento en la propia URL. El sistema busca esa referencia y devuelve un json con la información del evento y su respectivo recinto.
- **Operación de Descuento de Stock (POST /eventos/{id}/descontar)**
 - **Propósito:** Es una ruta de uso interno y transaccional. Permite que el módulo de reservas se comunique con el catálogo para restar un número exacto de entradas del aforo disponible.
 - **Intercambio de datos:** Recibe el identificador del evento en la URL y la cantidad de entradas deseadas en el cuerpo del mensaje. El sistema procesa la resta y devuelve un mensaje de éxito o un error indicando que no hay stock suficiente.

3.3.2 Interfaces de Gestión de Reservas

Encargada de recibir y gestionar las intenciones de compra.

- **Creación de Reserva (POST /reservas)**
 - **Propósito:** Es la puerta de entrada para que un usuario confirme que desea comprar entradas.
 - **Intercambio de datos:** Recibe un documento JSON con los datos fundamentales de la compra: quién es el cliente, qué evento desea, cuántas entradas solicita y el precio total. El sistema devuelve el recibo de la reserva indicando su estado final (CONFIRMADA o CANCELADA).
- **Visualización de la Reserva (Get /reservas/{id})**
 - **Propósito:** Permite consultar la información ampliada de una reserva específica.
 - **Intercambio de datos:** Recibe el identificador único de la reserva en la propia URL. El sistema busca esa referencia y devuelve un json con la información de esa reserva, con la información tanto del evento como del recinto, obteniéndola llamando al microservicio de eventos.

4. Desarrollo e Implementación

En este capítulo se detalla el proceso de construcción de la plataforma. A diferencia de un desarrollo tradicional centrado únicamente en escribir código, este proyecto se ha planteado desde el principio con un enfoque **Cloud-Native**. Esto significa que el diseño del software, su empaquetado en contenedores y la estrategia de despliegue en la nube se han planificado de forma conjunta.

4.1 Metodología de Desarrollo

Para no sumar complejidad a la infraestructura en la nube desde el primer día, el desarrollo se ha planteado de forma incremental e iterativa. La premisa ha sido validar cada pieza en entornos controlados antes de pasar a la siguiente fase.

1. **Diseño de Dominio y Arquitectura:** Antes de escribir código, se definieron los límites de cada microservicio siguiendo el **patrón Database-per-Service**, lo que garantiza que el catálogo y las reservas sean totalmente independientes a nivel de persistencia. Para mantener un código limpio y evitar duplicar lógica, se extrajeron los componentes comunes a dos módulos compartidos: un módulo domain (para las entidades, DTOs y excepciones de negocio) y un módulo shared-quarkus (para el control de excepciones).
2. **Desarrollo del código (Quarkus Dev Mode):** La implementación de la lógica de negocio se basó en el modo de desarrollo de **Quarkus**. Esta herramienta ha permitido validar rápidamente el mapeo con la base de datos, la inyección de dependencias y las solicitudes HTTP antes de generar las imágenes definitivas.
3. **Contenerización (Docker Compose):** Una vez validado el código, el siguiente paso fue empaquetar los microservicios mediante la creación de sus respectivos Dockerfile. Para probarlos en conjunto, se utilizó **Docker Compose** para levantar un entorno local y comprobar que los contenedores se comunicaban correctamente entre sí y que leían correctamente la configuración inyectada mediante variables de entorno.
4. **Pruebas en clúster local (Kubernetes):** Antes de desplegar en la nube pública, se sustituyó Docker Compose por un clúster local con Docker Desktop. En esta fase se redactaron los primeros **manifiestos YAML** (Deployments, Services y ConfigMaps), lo que permitió probar el comportamiento real de los Pods, la persistencia de los volúmenes y las capacidades de autorrecuperación (self-healing).

5. **Despliegue final en la nube (GKE):** La última etapa consistió en desplegar la arquitectura en Google Kubernetes Engine (GKE). Aquí se adaptó la infraestructura a un entorno real de producción: se integró la base de datos gestionada por **Cloud SQL** en sustitución de los contenedores locales, se configuró el acceso seguro mediante **Workload Identity** y se validó el comportamiento del clúster bajo carga.

Por último, para garantizar que todo este entorno sea reproducible y evitar configuraciones manuales, el proyecto se ha gestionado siguiendo el principio de Infraestructura como Código (IaC). Todos los manifiestos YAML y los archivos de configuración están bajo control de versiones en Git, lo que permite levantar la arquitectura completa desde cero de forma automatizada.

4.2 Arquitectura del Sistema

En este apartado se describe la topología técnica de la plataforma y el flujo de información. El diseño se basa en una arquitectura de microservicios desacoplados y aprovecha los servicios de Google Cloud Platform (GCP) para garantizar que el sistema sea tolerante a fallos y seguro.

Para entender mejor la arquitectura, la infraestructura se ha dividido en tres capas lógicas principales:

1. **Capa de Acceso y Enrutamiento:** Las peticiones externas entran al sistema a través de un **Cloud Load Balancer**, que proporciona una **IP pública** estable. Una vez que el tráfico llega al clúster, un controlador **Ingress** asume la función de enrutador. Este componente evalúa la ruta de la URL solicitada y dirige la petición al microservicio correspondiente (ws-reservas o ws-catalogo) de forma transparente para el usuario final.
2. **Capa de Lógica de Negocio (Clúster GKE):** El núcleo del sistema reside en Google Kubernetes Engine. Aquí, los microservicios se ejecutan de forma aislada en sus propios contenedores. Un punto clave de esta capa es la **comunicación inter-servicio**: por ejemplo, cuando un usuario solicita una compra, el servicio de reservas establece una comunicación síncrona vía HTTP con el catálogo para validar el aforo en tiempo real antes de consolidar la transacción.
3. **Capa de Persistencia (Database-per-Service):** Para evitar el acoplamiento de datos, cada microservicio cuenta con su propia base de datos **PostgreSQL** independiente, gestionada mediante **Cloud SQL**. Esta separación física y lógica garantiza que una alta carga de consultas en el catálogo de eventos no degrade el rendimiento de las operaciones de reserva.

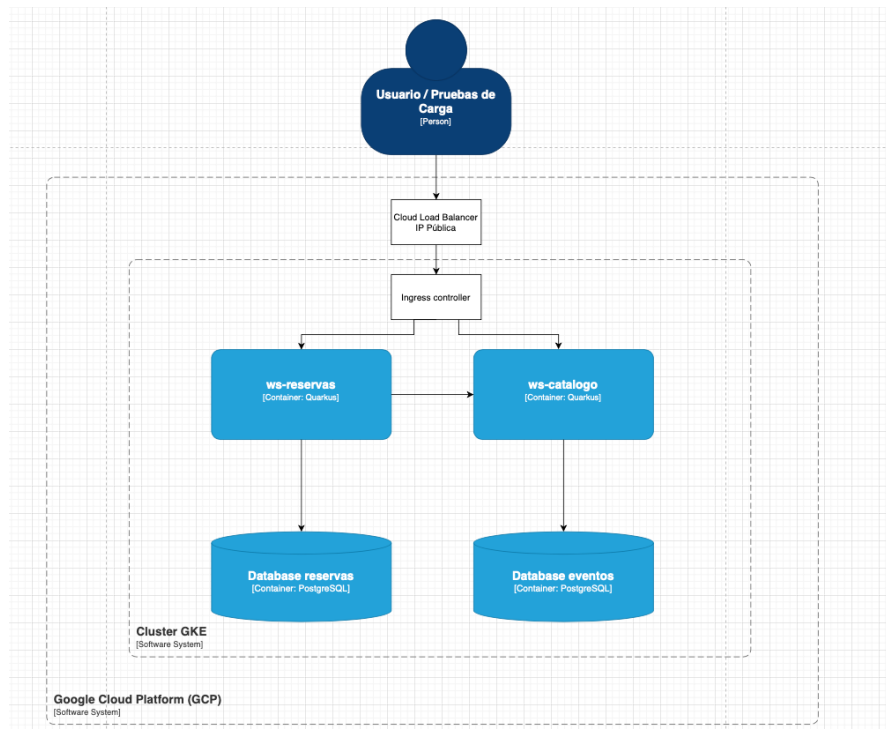


Figura 4-1 Diagrama de arquitectura para la plataforma de gestión de eventos

4.2.1 Seguridad y Patrón Sidecar

En materia de seguridad, una de las decisiones de diseño más importantes ha sido la implementación del **patrón Sidecar** para la comunicación con las bases de datos. En lugar de exponer Cloud SQL a internet o configurar direcciones IP autorizadas estáticas, se ha desplegado el **Cloud SQL Auth Proxy**. Este componente se ejecuta como un contenedor secundario que se inicia junto a cada microservicio dentro del mismo Pod, creando automáticamente un túnel cifrado y seguro hacia la base de datos.

Para complementar esto, se ha configurado la funcionalidad de **Workload Identity** en GKE, lo que permite trabajar con un enfoque Secretless (sin secretos estáticos). De este modo, los microservicios no necesitan tener contraseñas ni cadenas de conexión almacenadas en el código ni en variables de entorno. Simplemente asumen una identidad de servicio de Google Cloud (Service Account) para autenticarse de forma nativa contra la base de datos, lo que reduce drásticamente la superficie de exposición a posibles ataques.

4.3 Justificación del Stack Tecnológico

El diseño de un sistema distribuido exige elegir herramientas que no solo resuelvan la lógica de negocio, sino que también funcionen de manera eficiente en entornos de red. A continuación, se detallan las tecnologías elegidas y el motivo de su inclusión en la arquitectura:

4.3.1 Framework de Desarrollo: Quarkus

Para programar los microservicios se ha elegido **Quarkus**, un framework de Java optimizado específicamente para entornos de contenedores (Kubernetes-native). A diferencia de

los frameworks tradicionales de Java, que suelen penalizar el rendimiento con tiempos de arranque lentos y un alto consumo de RAM, Quarkus reduce drásticamente la huella de memoria. Esto resulta vital para el microservicio de reservas: ante un pico repentino de tráfico (como la salida a la venta de un evento muy demandado), Kubernetes necesita lanzar nuevos Pods en milisegundos. Quarkus permite que estos nuevos contenedores arranquen y empiecen a atender peticiones casi al instante.

4.3.2 Contenerización: Docker

La estandarización del entorno de ejecución se ha logrado mediante **Docker**. En una arquitectura de microservicios, es un reto asegurar que el software se comporte igual en el ordenador del desarrollador que en el servidor de producción. Docker soluciona esto encapsulando cada microservicio con sus dependencias exactas y su configuración base en una imagen inmutable. Esto elimina el clásico problema de "en mi máquina funciona" y permite probar la comunicación entre servicios en local antes de subir nada a la nube.

4.3.3 Orquestación y Cloud: Kubernetes y Google Kubernetes Engine (GKE)

Mientras Docker empaqueta la aplicación de forma estática, Kubernetes se encarga de gestionar esos contenedores en un entorno de producción dinámico. Su uso en este proyecto se justifica por tres características fundamentales:

- **Autocuración (Self-healing):** El clúster supervisa la salud de los Pods. Si un servicio falla o se bloquea, Kubernetes lo reinicia automáticamente sin intervención humana, garantizando la alta disponibilidad del sistema.
- **Descubrimiento de Servicios (Service Discovery):** Ofrece un DNS interno que permite a los microservicios encontrarse entre sí (por ejemplo, para que el servicio de reservas consulte el catálogo) sin importar qué IP tengan asignada ni en qué máquina física estén corriendo.
- **Escalabilidad Horizontal (Horizontal Pod Autoscaling):** Permite reaccionar a picos de demanda creando réplicas de un servicio de forma dinámica y destruyéndolas cuando la carga disminuye para ahorrar recursos.

4.3.4 Persistencia y Gestión de Base de Datos

Para mantener el principio de bajo acoplamiento, se ha aplicado el patrón **Database-per-Service con PostgreSQL**. Es un motor relacional robusto que garantiza la integridad de las transacciones, requisito indispensable para procesar reservas y gestionar el aforo del catálogo.

4.3.5 Elección del Proveedor Cloud: Google Cloud Platform (GCP)

Para alojar la infraestructura se ha optado por Google Cloud Platform. Tras evaluar alternativas como AWS o Microsoft Azure, se eligió la solución de Google por tres motivos prácticos:

1. **Integración nativa con Kubernetes:** Al ser Google el creador original de Kubernetes, su servicio gestionado (GKE) es uno de los más maduros y fáciles de integrar del mercado.
2. **Gestión simplificada:** Al delegar el mantenimiento del servidor base a Google, el esfuerzo de desarrollo se ha centrado íntegramente en la arquitectura de microservicios.
3. **Ecosistema cerrado:** El uso de **Cloud SQL** para las bases de datos y de **Artifact Registry** para las imágenes de Docker, ambos dentro de la misma red privada de Google, asegura una latencia mínima y un entorno protegido.

En definitiva, GCP ha ofrecido un buen equilibrio entre potencia técnica y facilidad de uso, lo que permite gestionar toda la infraestructura del proyecto de manera individual.

4.3.6 Control de Versiones y Repositorio: Git y GitHub

Para la gestión del código fuente se ha utilizado Git junto con **GitHub** como repositorio remoto. Esta decisión es indispensable para aplicar los principios de la Infraestructura como Código (IaC). En el repositorio no solo se guardan la lógica de negocio en Java, sino también los archivos Dockerfile y todos los manifiestos YAML de Kubernetes, lo que permite mantener versionada toda la evolución de la plataforma.

Todo el desarrollo realizado durante este Trabajo de Fin de Grado es de acceso público y se puede consultar en el siguiente repositorio:

- Repositorio: <https://github.com/vgarciacrespo/gestion-eventos-kubernetes>

4.4 Fases de Despliegue e Integración

4.4.1 Fase 1: Despliegue Base y Primer Microservicio

Antes de llevar la infraestructura a la nube, era necesario validar la arquitectura paso a paso. En ingeniería de software, intentar desplegar un sistema distribuido de golpe suele provocar errores de red difíciles de rastrear. Por eso, esta fase se planteó de forma incremental: desde el código en el entorno local hasta su ejecución aislada en contenedores.

El objetivo de esta primera etapa fue asegurar que el microservicio base fuera estable, inmutable y capaz de conectarse con su base de datos antes de añadir la complejidad de Kubernetes

4.4.1.1 Desarrollo y contenerización local

La programación del microservicio se realizó con Quarkus, en modo de desarrollo (lanzado mediante el comando `mvn quarkus:dev`). Esta herramienta permite realizar cambios en el código y ver el resultado en tiempo real, sin tener que reiniciar el servidor, lo que acelera muchísimo el desarrollo de la lógica de negocio y la creación de las entidades de la base de datos.

Con el código ya validado, el siguiente paso fue asegurar que la aplicación funcionara exactamente igual en cualquier entorno. Para lograr esta inmutabilidad, se empaquetó el servicio mediante la creación de un **Dockerfile**. El objetivo aquí no era solo meter el código en un contenedor, sino construir una imagen ligera y segura, utilizando compilaciones nativas para reducir el consumo de RAM y el tiempo de arranque.

Dado que un microservicio necesita su base de datos para funcionar, se utilizó **Docker Compose** para las pruebas locales. Esto permitió levantar en mi propio equipo el contenedor de la aplicación junto con un contenedor de **PostgreSQL**, comunicándolos entre sí a través de una red aislada.

Probar el sistema con Docker Compose en esta etapa fue clave por dos motivos:

- **Validación de variables de entorno:** Permitió comprobar que la aplicación leía la configuración (credenciales, puertos y URLs de conexión) desde fuera, evitando dejar datos sensibles estáticos o hardcodeados en el código.

- **Consistencia de datos:** Se verificó que los scripts de migración de la base de datos se ejecutaban correctamente al arrancar el contenedor, asegurando que el esquema de tablas estuviera listo antes de recibir peticiones.

Tras esta validación, el microservicio dejó de ser código en desarrollo para convertirse en una imagen funcional. A continuación, se muestra un extracto del archivo **docker-compose.yml** utilizado en local. En él se puede observar la inyección de variables (environment) y cómo se protege el arranque del microservicio asegurando que la base de datos esté lista primero, gracias a la directiva `depends_on` combinada con una sonda de salud (`service_healthy`):

```
ws-catalogo:
  build:
    context: ./ws-catalogo
    dockerfile: src/main/docker/Dockerfile.jvm
  container_name: ws-catalogo
  environment:
    DATABASE_HOST: database
    DATABASE_PORT: 5432
    DATABASE_NAME: upsa
    DATABASE_USERNAME: system
    DATABASE_PASSWORD: manager
  depends_on:
    database:
      condition: service_healthy
```

4.4.1.2 Prueba de Concepto en Orquestador Local

Una vez validada la aplicación en contenedores con Docker Compose, el siguiente paso fue migrar esa configuración a Kubernetes. Antes de asumir los costes y la complejidad de desplegar directamente en un proveedor de la nube pública, se optó por realizar una primera prueba en un entorno local.

Para ello, se habilitó un clúster de Kubernetes de un solo nodo que incluye Docker Desktop. Esta herramienta ofrece un entorno de pruebas totalmente funcional, perfecto para validar los primeros manifiestos YAML antes de pasarlos a producción.

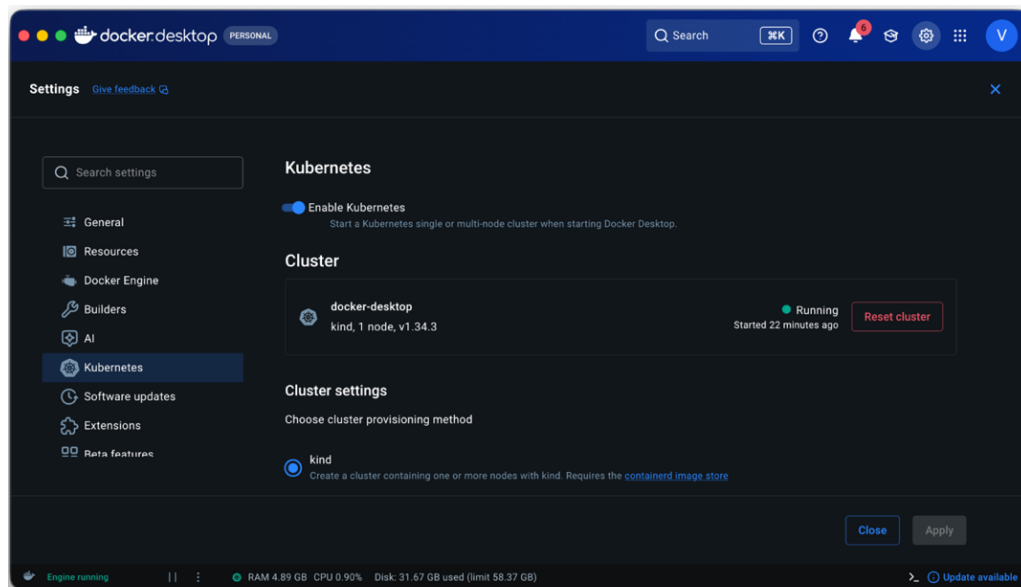


Figura 4-2 Activación del clúster local de Kubernetes integrado en Docker Desktop

La migración de la infraestructura a Kubernetes implicó diseñar las siguientes entidades:

- **Capa de Persistencia:** La base de datos pasó de ser un contenedor básico a definirse como un **StatefulSet**, lo que garantiza su identidad en la red y permite conectarla a un volumen persistente. Para inicializarla correctamente, el script SQL que crea las tablas se inyectó mediante un **ConfigMap** y las credenciales de acceso se aislaron en un **Secret** para evitar que las contraseñas quedaran expuestas en texto plano en el código.
- **Capa de Aplicación:** El microservicio desarrollado en Quarkus se empaquetó mediante un Deployment. En este punto se configuraron las sondas de salud (Liveness y Readiness Probes) para asegurar que Kubernetes solo envíe tráfico de red cuando el Pod esté completamente arrancado y listo para responder.
- **Conectividad:** La comunicación interna entre el microservicio y la base de datos se configuró creando entidades de tipo Service y aprovechando el DNS interno del clúster para que pudieran encontrarse dinámicamente.

Los manifiestos YAML resultantes de esta fase son los siguientes:

- ws-catalogo.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ws-catalogo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ws-catalogo
  template:
    metadata:
      labels:
        app: ws-catalogo
    spec:
      containers:
        - name: ws-catalogo
          image: gestion-eventos-ws-catalogo:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          livenessProbe:
            httpGet:
              path: /eventos/health/live
              port: 8080
            initialDelaySeconds: 15
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /eventos/health/ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          env:
            - name: DATABASE_HOST
              value: "eventos-db-service"
            - name: DATABASE_PORT
              value: "5432"
            - name: DATABASE_NAME
              value: "upsa"
            - name: DATABASE_USERNAME
              value: "system"
            - name: DATABASE_PASSWORD
```

```
        valueFrom:
          secretKeyRef:
            name: eventos-db-secret
            key: POSTGRES_PASSWORD
---
apiVersion: v1
kind: Service
metadata:
  name: ws-catalogo-service
spec:
  type: LoadBalancer
  selector:
    app: ws-catalogo
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

- postgres-db.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: eventos-db-secret
type: Opaque
stringData:
  POSTGRES_USER: system
  POSTGRES_PASSWORD: manager
  POSTGRES_DB: upsa
---
apiVersion: v1
kind: Service
metadata:
  name: eventos-db-service
spec:
  selector:
    app: eventos-db
  ports:
    - protocol: TCP
      port: 5432
      targetPort: 5432
  clusterIP: None
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: eventos-db
spec:
  serviceName: "eventos-db-service"
  replicas: 1
  selector:
    matchLabels:
      app: eventos-db
  template:
    metadata:
      labels:
        app: eventos-db
    spec:
      containers:
        - name: postgres
          image: postgres:15-alpine
          ports:
            - containerPort: 5432
          envFrom:
            - secretRef:
                name: eventos-db-secret
```

```

    volumeMounts:
      - name: db-data
        mountPath: /var/lib/postgresql/data
      - name: init-script
        mountPath: /docker-entrypoint-initdb.d/init.sql
        subPath: init.sql
    volumes:
      - name: init-script
        configMap:
          name: eventos-db-init
    volumeClaimTemplates:
      - metadata:
          name: db-data
        spec:
          accessModes: [ "ReadWriteOnce" ]
          resources:
            requests:
              storage: 1Gi

```

- postgres-configmap.yaml

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: eventos-db-init
data:
  init.sql: |
    init.sql: |

```

Para automatizar este proceso y seguir las **buenas prácticas de DevOps**, se creó un script de Bash llamado **deploy-local.sh**. Este script automatiza todos los pasos: compila el código con Maven, construye la **imagen de Docker** y aplica los **manifiestos YAML** en el clúster local en el orden correcto. Un detalle importante de este script es el uso del comando `kubectl rollout status statefulset/eventos-db`, que pausa la ejecución hasta que la base de datos esté totalmente levantada, evitando así que el microservicio intente conectarse antes de tiempo y falle.

```

# Construyendo la imagen Docker localmente
docker build -f ws-catalogo/src/main/docker/Dockerfile.jvm -t gestion-eventos-
ws-catalogo:latest ./ws-catalogo
#Desplegando la Base de Datos en Kubernetes
kubectl apply -f postgres-configmap.yaml
kubectl apply -f postgres-db.yaml
#Esperando a PostgreSQL
kubectl rollout status statefulset/eventos-db
#4. Despliegue ws-catalogo
kubectl apply -f ws-catalogo.yaml
# Verificar estado pods
kubectl get pods -w

```

```

configmap/eventos-db-init created
secret/eventos-db-secret created
service/eventos-db-service created
statefulset.apps/eventos-db created
Waiting for 1 pods to be ready...
partitioned roll out complete: 1 new pods have been updated...
deployment.apps/ws-catalogo created
service/ws-catalogo-service created

```

NAME	READY	STATUS	RESTARTS	AGE
eventos-db-0	1/1	Running	0	12s
ws-catalogo-6cf55f4f99-r2wqz	0/1	ContainerCreating	0	0s
ws-catalogo-6cf55f4f99-r2wqz	0/1	Running	0	2s
ws-catalogo-6cf55f4f99-r2wqz	1/1	Running	0	14s

Figura 4-3 Despliegue exitoso de la infraestructura con Kubernetes y estado final de los Pods en el entorno de pruebas local.

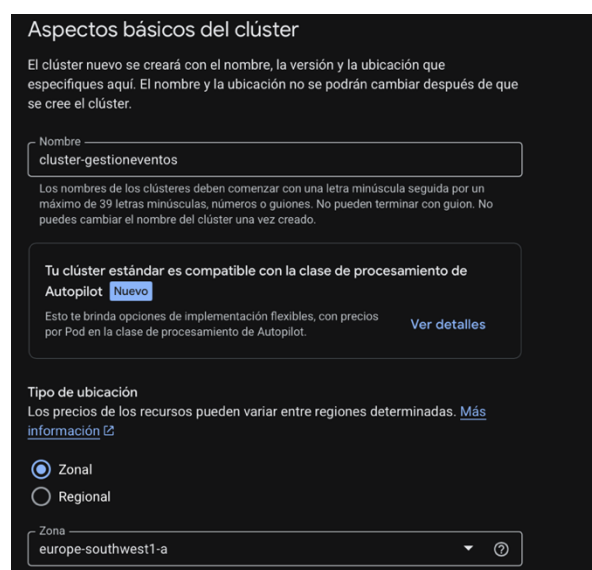
4.4.1.3 Preparación de la Infraestructura en la Nube (GCP)

Una vez validado el funcionamiento en local, la siguiente fase consistió en levantar la infraestructura real en la nube pública. Para ello, se utilizaron los servicios gestionados de Google Cloud Platform (GCP).

4.4.1.3.1 Despliegue del orquestador: Google Kubernetes Engine (GKE)

Se creó un clúster de Kubernetes en modo Standard llamado cluster-gestion-eventos. Se eligió el modo Standard en lugar de Autopilot porque permite al ingeniero tener un control total sobre el tipo de máquinas virtuales que componen los nodos, lo que ayuda a optimizar la infraestructura y los costes. Durante su creación, se tomaron las siguientes decisiones de configuración:

- **Latencia y ubicación (Región europe-southwest1-a):** El clúster se desplegó en el centro de datos de Madrid. Ubicar los servidores geográficamente cerca reduce considerablemente la latencia de red para los usuarios finales.



Aspectos básicos del clúster

El clúster nuevo se creará con el nombre, la versión y la ubicación que especifiques aquí. El nombre y la ubicación no se podrán cambiar después de que se cree el clúster.

Nombre
cluster-gestioneventos

Los nombres de los clústeres deben comenzar con una letra minúscula seguida por un máximo de 39 letras minúsculas, números o guiones. No pueden terminar con guion. No puedes cambiar el nombre del clúster una vez creado.

Tu clúster estándar es compatible con la clase de procesamiento de Autopilot **Nuevo**

Esto te brinda opciones de implementación flexibles, con precios por Pod en la clase de procesamiento de Autopilot. [Ver detalles](#)

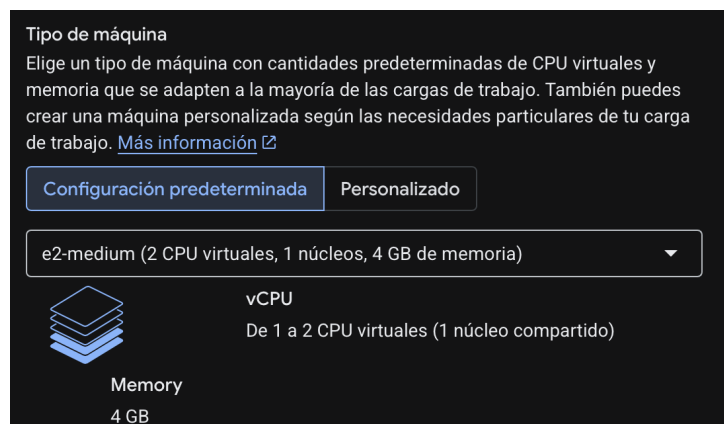
Tipo de ubicación
Los precios de los recursos pueden variar entre regiones determinadas. [Más información](#)

☒ Zonal
☐ Regional

Zona
europe-southwest1-a

Figura 4-4 Configuración zonal del clúster en la región de Madrid (europe-southwest1-a)

- **Ahorro de costes (Máquinas Spot):** Para los nodos de trabajo (Worker Nodes), se eligieron instancias e2-medium (2 vCPU, 4 GB de RAM) configuradas como máquinas Spot. Esto permite aprovechar recursos sobrantes de la nube de Google a un precio mucho menor.



Tipo de máquina

Elige un tipo de máquina con cantidades predeterminadas de CPU virtuales y memoria que se adapten a la mayoría de las cargas de trabajo. También puedes crear una máquina personalizada según las necesidades particulares de tu carga de trabajo. [Más información](#)

Configuración predeterminada Personalizado

e2-medium (2 CPU virtuales, 1 núcleo, 4 GB de memoria)

vCPU
De 1 a 2 CPU virtuales (1 núcleo compartido)

Memory
4 GB

Figura 4-5 Configuración de los nodos de Google Kubernetes Engine

- **Sistema Operativo (COS_CONTAINERD):** Los nodos se provisionaron con Container-Optimized OS, un sistema operativo muy ligero que Google diseña específicamente para ejecutar contenedores de forma segura y rápida.
- **Seguridad (Workload Identity):** Se activó esta funcionalidad en el clúster para que los Pods se comuniquen de forma segura con otros servicios. Esto evita tener que descargar y guardar contraseñas en texto plano, ya que la autenticación se hace mediante credenciales temporales gestionadas por el propio clúster.

4.4.1.3.2 Capa de Persistencia: Google Cloud SQL (PostgreSQL)

Para guardar los datos y el estado de los microservicios, se delegó la administración de la base de datos en el servicio Cloud SQL. La instancia, llamada gestion-eventos, se configuró con las siguientes características:

- **Motor de base de datos:** Se eligió PostgreSQL 18. Al usar la versión gestionada por Google, se obtienen mejoras de rendimiento automáticas y un almacenamiento en caché optimizado.
- **Capacidad y rendimiento:** Para simular un entorno de producción real capaz de soportar picos de concurrencia, se levantó una máquina potente (db-perf-optimized-N-8), con 8 vCPU, 64 GB de RAM y 250 GB de disco SSD.
- **Ubicación y copias de seguridad:** Al igual que el clúster, la base de datos se desplegó en Madrid (europe-southwest1) para que la comunicación interna fuera lo más rápida posible. Para asegurar los datos ante posibles fallos, se activaron las copias de seguridad automáticas, lo que permite restaurar la base de datos a un punto exacto en el tiempo. Es importante destacar que, para los propósitos de esta prueba de concepto y debido a las restricciones presupuestarias en un entorno académico, la instancia se ha aprovisionado en una única zona (europe-southwest1-a). Sin embargo, la arquitectura está diseñada para soportar una alta disponibilidad real. En un entorno de producción corporativo, bastaría con habilitar la opción de "Alta Disponibilidad (Regional)" en Google Cloud, lo que desplegaría automáticamente una réplica en standby en otra zona sin requerir ningún cambio en el código ni en los manifiestos de Kubernetes.
- **Conectividad y patrón Sidecar:** A nivel de red, se asignó a la base de datos una IP pública para facilitar las labores de depuración y la conexión directa durante la fase de desarrollo. Pero es importante entender que esto no compromete la seguridad del clúster, ya que las conexiones externas están bloqueadas por el firewall de GCP y el tráfico desde los Pods se realiza exclusivamente a través del Cloud SQL Auth Proxy. Este componente se ejecuta como un túnel cifrado (patrón Sidecar) combinado con Workload Identity, delegando la seguridad en Google y haciendo que la base de datos rechace cualquier conexión que no provenga directamente del clúster autorizado. Como mejora de seguridad (Hardening) para un paso definitivo a producción, sería mejor deshabilitar la IP pública y configurar el acceso exclusivamente mediante direcciones IP privadas dentro de la VPC.
- **Inicialización del esquema:** Para la creación inicial de las tablas y secuencias en la instancia de Cloud SQL, se optó por ejecutar los scripts SQL manualmente a través de la interfaz Cloud SQL Studio integrada en la consola de Google Cloud. En una transición a un entorno de producción corporativo, la gestión y la evolución del esquema de la base de datos se automatizarían mediante herramientas de migración como Flyway o Liquibase. Esta integración es vital para garantizar que los despliegues automáticos sin cortes (Rolling Updates) de Kubernetes mantengan la

retrocompatibilidad, evitando que los Pods de una versión anterior fallen al convivir temporalmente con un nuevo esquema.

Resumen	
Edición de Cloud SQL	Enterprise Plus
Región	europe-southwest1 (Madrid)
Versión de la base de datos	PostgreSQL 18
Tipo de máquina	db-perf-optimized-N-8
CPU virtuales	8 vCPU
RAM	64 GB
Caché de datos	Habilitada (375 GB)
Almacenamiento	250 GB SSD
Conexiones	IP pública
Copia de seguridad	Automatizada
Disponibilidad	Zona única
Recuperación de un momento determinado	Habilitada
Capacidad de procesamiento de la red (MB/s)	2,000 de 2,000
IOPS	Lectura: 13,500 de 15,000 Escritura: 13,500 de 15,000
Capacidad de procesamiento del disco (MB/s)	Lectura: 360.0 de 800.0 Escritura: 360.0 de 800.0

Figura 4-6 Resumen de configuración de la instancia de Cloud SQL

4.4.1.4 Despliegue de los microservicios en GKE

Con la infraestructura y la base de datos ya listas en Google Cloud, el siguiente paso fue empaquetar y desplegar el primer microservicio: el Catálogo de Eventos (ws-catalogo). Este proceso se dividió en tres partes: contenerización, configuración de los manifiestos y automatización.

4.4.1.4.1 Contenedorización y Almacenamiento (Artifact Registry)

Para que Kubernetes pueda arrancar los contenedores en la nube, las imágenes deben estar almacenadas en un repositorio al que el clúster tenga acceso. Para ello se utilizó Google Artifact Registry.

Primero, se compiló el código fuente con la versión de la JVM de Quarkus. Después, la imagen se etiquetó con la ruta del repositorio remoto ubicado en Madrid y se subió mediante Docker. Los comandos utilizados para este proceso fueron los siguientes:

```
#imagen docker
docker build -f ws-catalogo/src/main/docker/Dockerfile.jvm -t gestion-eventos-
ws-catalogo:latest ./ws-catalogo

#tagear imagen
docker tag gestion-eventos-ws-catalogo:latest europe-southwest1-
docker.pkg.dev/gestionreservaseventos/repo-eventos/ws-catalogo:v5

#subir imagen
docker push europe-southwest1-docker.pkg.dev/gestionreservaseventos/repo-
eventos/ws-catalogo:v5
```

4.4.1.4.2 Configuración Declarativa: Despliegue y Patrón Sidecar

Una vez que la imagen está subida al registro privado, el despliegue en Google Kubernetes Engine (GKE) se realiza mediante un manifiesto en formato YAML. En lugar de desplegar cada cosa por separado, se agrupó toda la configuración en un único archivo (ws-catalogo-cloud.yaml) que define exactamente cómo debe comportarse el microservicio en producción.

A continuación se muestra el contenido del manifiesto:

- Manifiesto ws-catalogo-cloud.yaml

```

apiVersion: v1
kind: Secret
metadata:
  name: eventos-db-secret
type: Opaque
stringData:
  POSTGRES_PASSWORD: "TU_CONTRASEÑA_AQUI"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ws-catalogo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ws-catalogo
  template:
    metadata:
      labels:
        app: ws-catalogo
    spec:
      serviceAccountName: ksa-backend
      containers:
        - name: ws-catalogo
          image: europe-southwest1-docker.pkg.dev/gestionreservaseventos/repo-
eventos/ws-catalogo:v5
          imagePullPolicy: Always
          ports:
            - containerPort: 8080
          livenessProbe:
            httpGet:
              path: /eventos/health/live
              port: 8080
              initialDelaySeconds: 15
              periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /eventos/health/ready
              port: 8080
              initialDelaySeconds: 5
              periodSeconds: 10
          env:
            - name: DATABASE_HOST
              value: "127.0.0.1"
            - name: DATABASE_PORT
              value: "5432"
            - name: DATABASE_NAME
              value: "eventos-db"
            - name: DATABASE_USERNAME
              value: "postgres"
            - name: DATABASE_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: eventos-db-secret
                  key: POSTGRES_PASSWORD
            - name: cloud-sql-proxy
              image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.8.1
              args:
                - "--port=5432"
                - "gestionreservaseventos:europe-southwest1:gestion-eventos"

```

```
    securityContext:
      runAsNonRoot: true
---
apiVersion: v1
kind: Service
metadata:
  name: ws-catalogo-service
spec:
  type: LoadBalancer
  selector:
    app: ws-catalogo
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

4.4.1.4.2.1 *Análisis y Decisiones del Diseño del Despliegue*

Este archivo YAML no es solo una lista de contenedores, sino que aplica varias decisiones clave de arquitectura:

- **Gestión Segura de Credenciales (Secret):** Siguiendo el principio de separar el código de la configuración, el manifiesto define primero un objeto de tipo Secret. Aquí se guarda la contraseña de la base de datos de forma oculta, evitando que quede expuesta en texto plano en el repositorio de código o en la imagen del contenedor.
- **Despliegue y Sondas de Salud (Deployment):** Se define un controlador de tipo Deployment que orquesta la ejecución. A este controlador se le han configurado sondas de salud (Liveness y Readiness Probes) que apuntan a las rutas nativas de Quarkus (/eventos/health/live y /ready). Esto permite al clúster saber cuándo el Pod está listo para recibir tráfico y le otorga la capacidad de ejecutar tareas de autorrecuperación (Self-healing) reiniciando el contenedor si este deja de responder.
- **Inyección Dinámica de Configuración:** Los datos de conexión y las credenciales almacenadas en el Secret se inyectan dinámicamente en el contenedor principal como variables de entorno (mediante la directiva valueFrom). Esto asegura que la aplicación tenga sus datos listos en tiempo de ejecución, manteniendo intacta la inmutabilidad de la imagen Docker.
- **El patrón Sidecar y la Seguridad de Red (Cloud SQL Auth Proxy):** Para conectar el microservicio con la base de datos administrada sin exponerla a internet, se implementó el patrón Sidecar. Dentro del mismo Pod se levanta un contenedor secundario (cloud-sql-proxy). Como los contenedores de un mismo Pod comparten la red, el proxy abre el puerto 5432 en el bucle de retorno (loopback). Así, el microservicio de Java simplemente se conecta a 127.0.0.1:5432 como si la base de datos estuviera en local. El proxy intercepta esa conexión, la cifra y la envía de forma segura a través de la red privada (backbone) de Google Cloud hasta la instancia real.
- **Workload Identity y Autenticación sin claves:** Para que el proxy tenga permisos para acceder a la base de datos, el Pod asume la cuenta de servicio de Kubernetes ksa-backend. Gracias a la característica Workload Identity de GKE, esta cuenta se vincula directamente a una cuenta de IAM de Google Cloud. Esto permite que el contenedor se autentique de forma nativa sin tener que generar, guardar ni rotar archivos de contraseñas manualmente, lo que reduce drásticamente la superficie de ataque.

Por último, el archivo define un **Service** de tipo **LoadBalancer**. Esto le indica al controlador de Google Cloud que debe aprovisionar un balanceador de carga físico y asignarle una dirección IP pública estable para que el microservicio reciba tráfico externo.

4.4.1.4.3 Automatización del Despliegue

Para no tener que escribir los comandos a mano cada vez, se creó un script automatizado (deploy-cloud.sh). Este proceso sigue tres pasos:

- **Aprovisionamiento dinámico de nodos:**

Como el clúster está configurado para escalar a 0 nodos cuando no se usa (para ahorrar costes durante el desarrollo), el primer comando obliga a crear una máquina virtual en Madrid.

```
# Arrancar el clúster en Google Cloud
gcloud container clusters resize cluster-gestion-eventos --num-nodes=1 --zone europe-southwest1-a
```

- **Autenticación y obtención de contexto:**

Descarga los certificados necesarios para que el ordenador local pueda comunicarse de forma segura con el clúster en la nube.

```
#Conectar terminal a la nube
gcloud container clusters get-credentials cluster-gestion-eventos --region europe-southwest1-a
```

- **Aplicación del manifiesto declarativo:**

Envía el archivo YAML a Kubernetes para que proceda a iniciar los contenedores.

```
kubectl apply -f ws-catalogo-cloud.yaml
```

Para comprobar que todo se levanta correctamente, se ejecuta el comando `kubectl get pods -w`, que permite ver en tiempo real cómo el Pod atraviesa los distintos estados hasta llegar a Running.

```
kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
ws-catalogo-55f769cc66-rbsz2	0/2	ContainerCreating	0	1s
ws-catalogo-55f769cc66-rbsz2	1/2	Running	0	5s
ws-catalogo-55f769cc66-rbsz2	2/2	Running	0	18s

Figura 4-7 . Verificación en tiempo real del arranque del Pod

4.4.1.4.4 Verificación y Acceso Externo

Una vez terminado el despliegue, el balanceador de carga de Google Cloud asigna una IP pública al sistema. Ejecutando el comando `kubectl get svc`, se comprueba la IP exacta que se le ha otorgado para acceder al puerto 8080.

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP	28d
ws-catalogo-service	LoadBalancer	34.118.233.167	34.175.190.51	8080:32727/TCP	28d

Figura 4-8 Asignación de dirección IP pública al microservicio mediante el balanceador de carga de Google Kubernetes Engine

Para validar que la aplicación funciona y que realmente se conecta a la base de datos mediante el túnel Sidecar, se abre un navegador y se accede a esa IP pública (<http://34.175.190.51:8080/eventos/swagger-ui/>). El resultado es la interfaz de Swagger autogenerada por Quarkus. Ver esta pantalla confirma que el contenedor funciona

correctamente y permite probar en tiempo real los endpoints creados para la gestión de eventos.

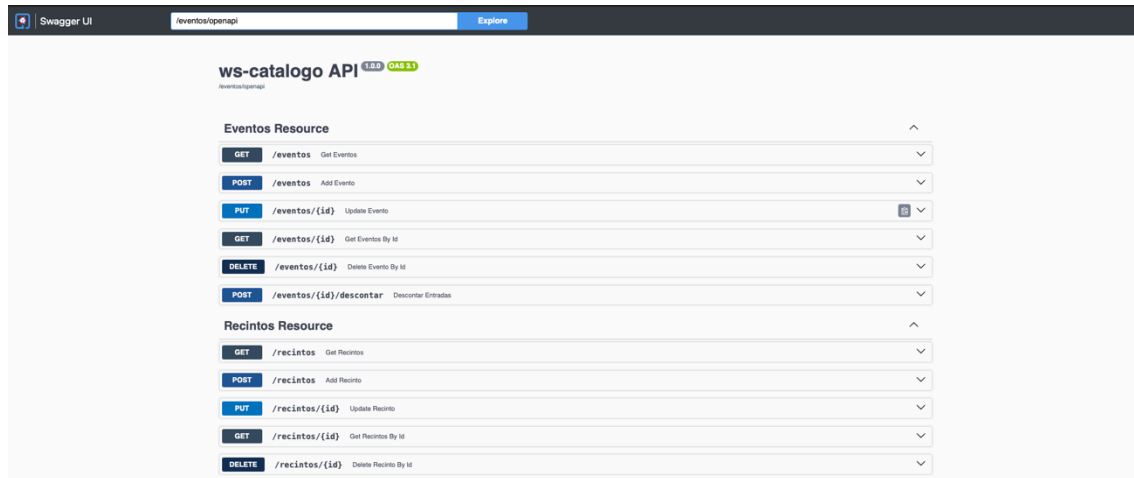


Figura 4-9 Interfaz Swagger UI del microservicio de catálogo accesible públicamente

4.4.2 Fase 2: Evolución a Arquitectura Distribuida

4.4.2.1 Evolución de la Arquitectura: Del entorno local a la nube pública

Tras validar el funcionamiento del primer componente en la Fase 1, el siguiente paso fue dividir el sistema en microservicios independientes y migrar la arquitectura completa a la nube pública.

Para no romper el sistema en el proceso y asegurar la estabilidad, esta fase se planteó de forma escalonada:

1. Validación Local: Primero, los microservicios de Catálogo y Reservas se probaron en conjunto en el clúster local (Docker Desktop). Esto permitió depurar y comprobar la comunicación entre ellos sin incurrir en gastos de recursos de la nube.

2. Aislamiento Lógico de Datos: Siguiendo el patrón Database-per-Service, se dividió la base de datos. Para no multiplicar los costes creando varios servidores físicos, se optó por crear bases de datos lógicas separadas (eventos-db y reservas-db) dentro de la misma instancia gestionada de Google Cloud SQL.

3. Despliegue Distribuido (GKE): Por último, las imágenes ya probadas se subieron a Artifact Registry y se desplegaron en Google Kubernetes Engine (GKE), dejando que el clúster se encargara de conectarlas entre sí mediante su sistema interno (Service Discovery) y de balancear el tráfico hacia el exterior.

A lo largo de los siguientes apartados se explican las decisiones de diseño, los manifiestos creados y cómo se solucionó la comunicación interna para que este entorno distribuido funcionara correctamente.

4.4.2.2 Segregación de Datos: Patrón Database-per-Service

En una arquitectura de microservicios es importante mantener un bajo acoplamiento. Esto significa que cada servicio debe ser dueño de su propio almacenamiento y gestionarlo. Al separar la lógica del Catálogo de la de las Reservas, era obligatorio dividir también sus datos para evitar el clásico problema de la "base de datos compartida", un antipatrón que genera cuellos de botella y rompe la independencia al desplegar nuevas versiones.

Para aplicar este patrón en Google Cloud, se plantearon dos opciones de infraestructura:

- **Aislamiento Físico:** Consiste en crear un servidor de Cloud SQL independiente para cada microservicio. Aunque ofrece el máximo nivel de aislamiento, duplica los costes fijos de computación, de almacenamiento y de mantenimiento. Para el alcance y el volumen de tráfico de este proyecto, era una opción ineficiente y muy costosa.
- **Aislamiento Lógico:** Consiste en aprovechar la instancia de PostgreSQL ya creada (gestion-eventos) y crear dentro de ella bases de datos lógicamente separadas, cada una asociada a un usuario con permisos restringidos exclusivamente a sus propias tablas.

Se optó por el aislamiento lógico. Esta decisión cumple a la perfección con la teoría del patrón arquitectónico (el catálogo y las reservas no comparten tablas ni pueden leer los datos del otro) y, al mismo tiempo, optimiza los recursos y mantiene controlada la factura de Google Cloud.

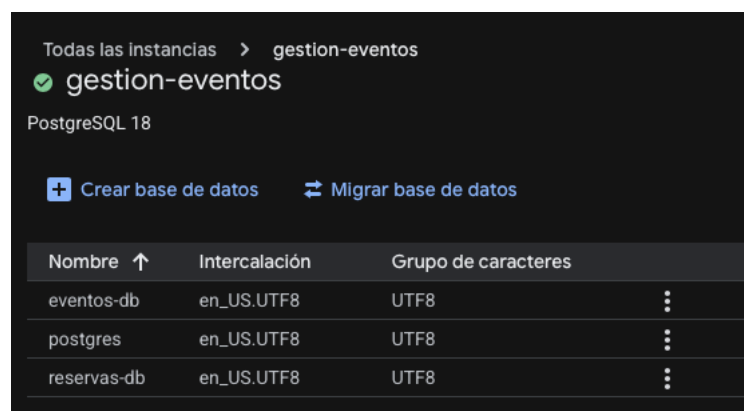


Figura 4-10 Consola de Google Cloud SQL mostrando las bases de datos lógicas de cada microservicio coexistiendo en la misma instancia gestion-eventos

4.4.2.3 Contenerización y Registro en la Nube (Google Artifact Registry)

4.4.2.3.1 Construcción y versionado de imágenes Docker

Tras separar lógicamente las bases de datos, el siguiente paso fue empaquetar el código de los microservicios Catálogo (ws-catalogo) y Reservas (ws-reservas) en contenedores independientes. Como ambos servicios están desarrollados con Quarkus, se utilizó su configuración optimizada para la máquina virtual de Java (JVM), lo que permite generar imágenes ligeras y de arranque rápido en el clúster.

Este proceso de empaquetamiento y subida al repositorio privado se divide en cuatro pasos:

1. **Compilación y empaquetamiento de la aplicación:** Antes de crear el contenedor, se realiza una compilación limpia del código Java con Maven.

```
mvn clean package
```

2. **Construcción de las imágenes (Docker Build):** A partir del archivo Dockerfile.jvm proporcionado por el propio entorno de Quarkus, se construyen las imágenes locales de ambos microservicios.

```
docker build -f ws-catalogo/src/main/docker/Dockerfile.jvm -t gestion-eventos-  
ws-catalogo:latest ws-catalogo  
docker build -f ws-reservas/src/main/docker/Dockerfile.jvm -t gestion-eventos-  
ws-reservas:latest ws-reservas
```

- 3. Etiquetado y versionado (Docker Tag):** Para que Google Artifact Registry acepte las imágenes, es necesario etiquetarlas según su formato oficial. Esta ruta incluye la región (europe-southwest1), el nombre del proyecto en Google Cloud, el repositorio y la versión de la imagen (v1).

```
docker tag gestion-eventos-ws-catalogo:latest europe-southwest1-  
docker.pkg.dev/gestionreservaseventos/repo-eventos/ws-catalogo:v1  
docker tag gestion-eventos-ws-reservas:latest europe-southwest1-  
docker.pkg.dev/gestionreservaseventos/repo-eventos/ws-reservas:v1
```

- 4. Publicación en la nube (Docker Push):** Tras autenticar la terminal con las credenciales de Google Cloud, se suben las imágenes al repositorio privado. Desde allí, los nodos del clúster de Kubernetes podrán descargarlas internamente de forma rápida y segura durante el despliegue.

```
docker push europe-southwest1-  
docker.pkg.dev/gestionreservaseventos/repo-eventos/ws-catalogo:v1  
docker push europe-southwest1-  
docker.pkg.dev/gestionreservaseventos/repo-eventos/ws-reservas:v1
```

Este proceso garantiza la inmutabilidad: el código que se ejecutará en los servidores de Google será exactamente el mismo que acaba de compilarse, evitando cualquier inconsistencia o problema de dependencias al pasar a producción.

4.4.2.4 *Despliegue y Orquestación en Google Kubernetes Engine (GKE)*

Con las imágenes ya disponibles en el registro privado, el despliegue en el clúster se realiza mediante manifiestos YAML. Igual que en la fase anterior, toda la configuración de despliegue, de red y de secretos se abstrae en archivos declarativos.

A continuación se muestra el manifiesto unificado diseñado para desplegar el microservicio de Reservas (ws-reservas). Este archivo agrupa el secreto para la conexión a su base de datos independiente, la definición del Pod (Deployment) y la configuración de acceso por red (Service):

El manifiesto empleado para la gestión de eventos (ws-catalogo) es el mismo que usamos en la fase 1 ([Configuración Declarativa: Despliegue y Patrón Sidecar](#)).

- Manifiesto ws-catalogo-cloud.yaml

```
apiVersion: v1  
kind: Secret  
metadata:  
  name: reservas-db-secret  
type: Opaque  
stringData:  
  POSTGRES_PASSWORD: "TU_CONTRASEÑA_AQUI"  
---  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: ws-reservas  
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: ws-reservas  
  template:
```

```

metadata:
  labels:
    app: ws-reservas
spec:
  serviceAccountName: ksa-backend
  containers:
    - name: ws-reservas
      image: europe-southwest1-docker.pkg.dev/gestionreservaseventos/rep-
eventos/ws-reservas:v1
      imagePullPolicy: Always
      ports:
        - containerPort: 8080
      livenessProbe:
        httpGet:
          path: /reservas/health/live
          port: 8080
          initialDelaySeconds: 15
          periodSeconds: 10
      readinessProbe:
        httpGet:
          path: /reservas/health/ready
          port: 8080
          initialDelaySeconds: 5
          periodSeconds: 10
      env:
        - name: DATABASE_HOST
          value: "127.0.0.1"
        - name: DATABASE_PORT
          value: "5432"
        - name: DATABASE_NAME
          value: "reservas-db"
        - name: DATABASE_USERNAME
          value: "postgres"
        - name: DATABASE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: reservas-db-secret
              key: POSTGRES_PASSWORD
        - name: SERVICE_EVENTOS_URL
          value: "http://ws-catalogo-service:8080"
    - name: cloud-sql-proxy
      image: gcr.io/cloud-sql-connectors/cloud-sql-proxy:2.8.1
      args:
        - "--port=5432"
        - "gestionreservaseventos:europe-southwest1:gestion-eventos"
      securityContext:
        runAsNonRoot: true
---
apiVersion: v1
kind: Service
metadata:
  name: ws-reservas-service
spec:
  type: LoadBalancer
  selector:
    app: ws-reservas
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080

```

4.4.2.4.1 Análisis de Diseño del Despliegue

Este manifiesto aplica varias decisiones de diseño fundamentales para que el sistema sea seguro y tolerante a fallos:

- **Gestión Segura de Credenciales (Kubernetes Secrets):** Para no dejar la contraseña de la base de datos en el código, se utiliza el recurso Secret (reservas-db-secret). Al declararlo como tipo Opaque, Kubernetes cifra la contraseña en su almacén interno (etcd). Al arrancar el Pod, la directiva valueFrom.secretKeyRef recupera la contraseña y la inyecta como variable de entorno DATABASE_PASSWORD directamente en el contenedor, evitando exponerla en texto plano. Si bien esta aproximación es el estándar para demostrar el funcionamiento interno de Kubernetes en un entorno académico, en un contexto de producción real sería necesario integrar un gestor de claves externo, como Google Secret Manager o HashiCorp Vault, para garantizar el cifrado avanzado en reposo y evitar que los valores queden expuestos en los manifiestos YAML.
- **Conexión Segura mediante el Patrón Sidecar (Cloud SQL Auth Proxy):** Para conectar el microservicio a la base de datos sin exponer Cloud SQL a internet, se utiliza el patrón Sidecar. El manifiesto indica que el Pod va a levantar dos contenedores. Como ambos comparten la misma red local, el contenedor secundario (cloud-sql-proxy) crea un túnel cifrado. Así, la aplicación principal en Quarkus simplemente se conecta a 127.0.0.1 (localhost) de forma nativa, y el proxy se encarga de enviarlo a Google Cloud autenticándose automáticamente con la cuenta de servicio ksa-backend.
- **Control de Salud y Disponibilidad (Probes):** Kubernetes vigila constantemente que la aplicación no se quede colgada mediante las directivas livenessProbe y readinessProbe. Estas sondas consultan las rutas de salud nativas de Quarkus (/health/live y /health/ready). Gracias a la sonda de disponibilidad (Readiness), el balanceador de carga no envía tráfico de usuarios al contenedor hasta que este confirme que ha logrado conectarse a la base de datos, lo que garantiza despliegues sin interrupción de servicio (Zero-Downtime Deployment).
- **Acceso desde el Exterior (Service LoadBalancer):** Por último, se configura un recurso Service de tipo LoadBalancer. Esto ordena a Google Cloud que aprovisiona un balanceador de carga físico con una IP pública estable, que se encargue de recibir el tráfico externo en el puerto 8080 y de distribuirlo de forma uniforme entre los contenedores internos.

4.4.2.5 Ejecución y Validación del Despliegue en GKE

Con los manifiestos listos, el siguiente paso fue aplicar toda esta configuración al clúster.

Primero, se autenticó la terminal local contra el clúster de Google Cloud para descargar las credenciales de acceso:

```
gcloud container clusters get-credentials cluster-gestion-eventos --region europe-southwest1-a
```

Después, se aplicaron los manifiestos mediante el comando kubectl. Esto hace que Kubernetes inicie todo el proceso automáticamente: descarga las imágenes desde Artifact Registry, crea los secretos y levanta los Pods:

```
kubectl apply -f f2-kubernetes-cloud/ws-catalogo.yaml
kubectl apply -f f2-kubernetes-cloud/ws-reservas.yaml
```

Para comprobar que los contenedores de Quarkus y los proxies de Cloud SQL arrancaban correctamente, se monitorizó el estado de los Pods en tiempo real hasta verificar que ambos contenedores dentro de cada Pod estaban operativos (2/2 Running):

```
kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS
ws-catalogo-9d6d456f6-f42h9	2/2	Running	0
ws-reservas-f9699b67-7z7kt	2/2	Running	0

Figura 4-11 Estado de los Pods (aplicación + proxy sidecar)

Una vez levantados los Pods, se consultó el estado de los servicios de red para obtener las direcciones IP públicas asignadas por los balanceadores de carga de Google Cloud:

```
kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP
ws-catalogo-service	LoadBalancer	34.118.231.126	34.175.190.51	8080:30571/TCP
ws-reservas-service	LoadBalancer	34.118.234.228	34.175.98.65	8080:31937/TCP

Figura 4-12 Asignación de IP pública mediante Load Balancer

4.4.2.6 Validación de las APIs Distribuidas mediante Swagger UI

Con las direcciones IP públicas obtenidas, se verificó el acceso real al sistema. Al acceder a través de un navegador web al puerto 8080 de cada IP, los microservicios sirven automáticamente su interfaz gráfica, generada automáticamente por Swagger UI.

- **Interfaz de Catálogo:** Disponible a través del balanceador del primer microservicio (<http://34.175.190.51:8080/eventos/swagger-ui/>).
- **Interfaz de Reservas:** Disponible de forma independiente a través del segundo balanceador (<http://34.175.98.65:8080/reservas/swagger-ui/>).

Como se ve en las Figuras 4-13 y 4-14, ver estas interfaces operativas no solo confirma que los servicios están funcionando y recibiendo tráfico a través de internet, sino que también muestra de forma interactiva todo el contrato de la API. El hecho de que Swagger esté expuesto permite que cualquier sistema cliente realice pruebas de integración reales (peticiones GET, POST, etc.) directamente desde su navegador, abstrayendo toda la complejidad de la infraestructura subyacente en Kubernetes.

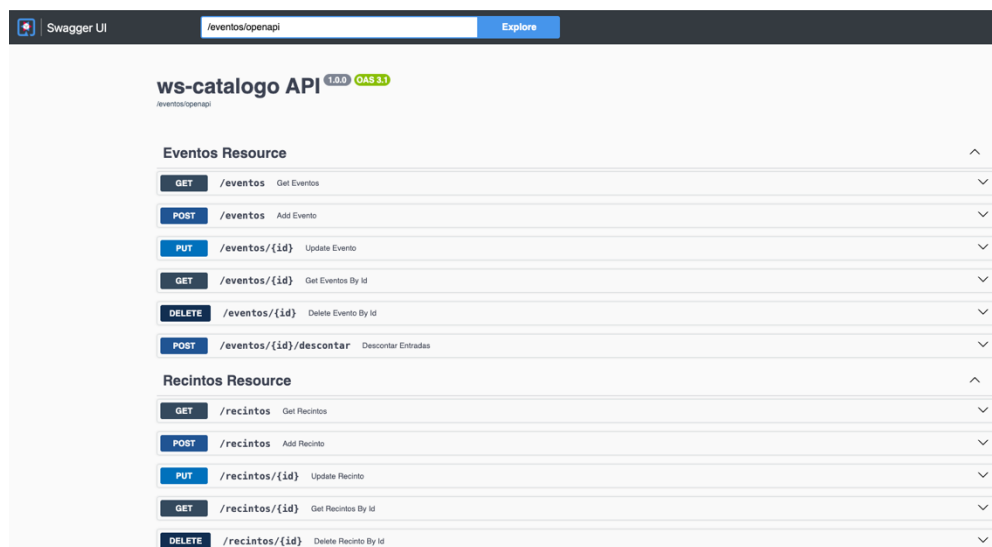


Figura 4-13 Interfaz interactiva de Swagger UI para el microservicio ws-catalogo desplegado en GKE

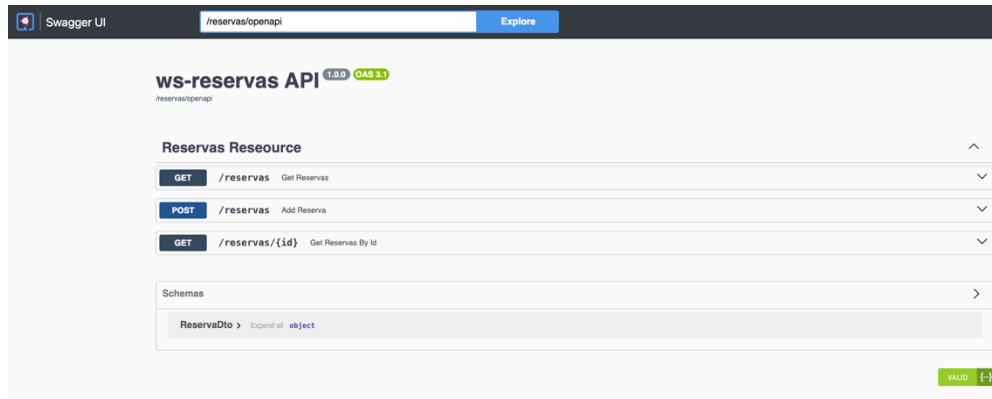


Figura 4-14 Interfaz interactiva de Swagger UI para el microservicio ws-reservas desplegado en GKE

4.4.2.7 Validación de la Comunicación Inter-servicios (Pruebas de Integración)

La última prueba de la Fase 2 consistió en verificar que el tráfico interno entre los microservicios se comunicaba y se enrutaba correctamente, validando así el funcionamiento del patrón de **Service Discovery** en Kubernetes.

Para demostrar esta conectividad, se diseñó una prueba de integración mediante el envío de una petición HTTP desde la interfaz de Swagger UI al microservicio de reservas, lo que requería la colaboración en tiempo real entre ambos servicios.

El flujo técnico que se ejecuta por debajo al hacer la petición es el siguiente:

- **Entrada del tráfico:** La petición llega desde el exterior a través del balanceador de carga público asociado a **ws-reservas**.
- **Resolución DNS interna:** Al necesitar validar los datos del evento, el microservicio de reservas realiza una llamada HTTP mediante su cliente REST, apuntando al nombre del servicio de catálogo (`http://ws-catalogo-service:8080`). El servidor CoreDNS del clúster intercepta la llamada y traduce ese nombre de servicio directamente a la IP privada del Pod de catálogo.
- **Procesamiento distribuido:** El microservicio de catálogo recibe la petición, consulta su base de datos lógica (eventos-db) y devuelve los datos al servicio de reservas, que finaliza la transacción en su propio esquema (reservas-db).

Como se muestra en la **Figura 4-15**, la respuesta con el código de estado HTTP 200 confirma que la comunicación interna entre los Pods funciona a la perfección, lo que valida por completo la arquitectura distribuida de esta fase.

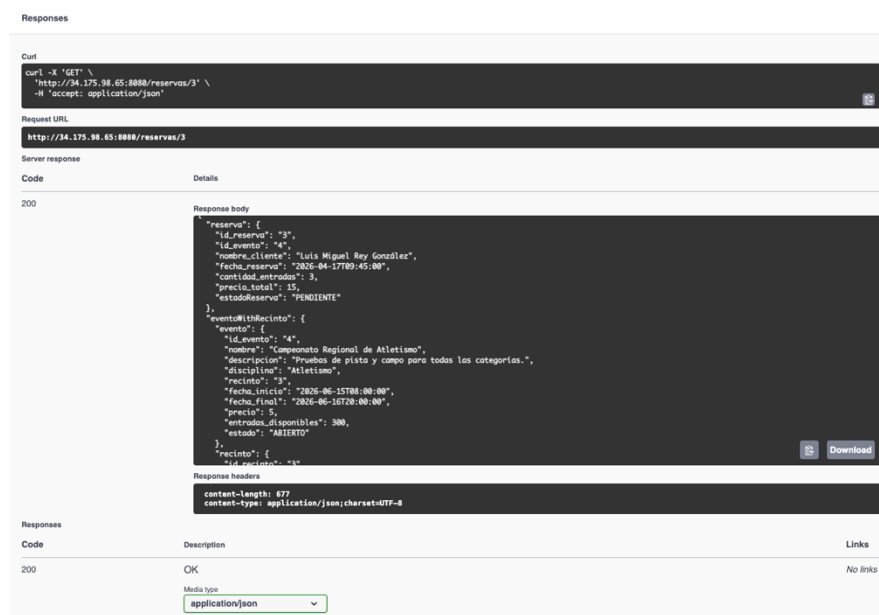


Figura 4-15 Prueba de integración demostrando la comunicación síncrona entre el microservicio de reservas y el de catálogo a través del DNS interno de Kubernetes

4.4.3 Evolución hacia la Producción: Unificación mediante Ingress Controller

En las fases anteriores de validación, cada microservicio se expuso mediante un servicio de tipo LoadBalancer independiente. Aunque este enfoque facilita las pruebas aisladas, en un entorno de producción real no es recomendable, ya que Google Cloud cobra por cada IP pública asignada y, además, el sistema carece de un punto de entrada único (API Gateway) que centralice el tráfico y las políticas de seguridad.

Para solucionar esto y mejorar la infraestructura, se rediseñó la capa de red mediante la implementación de un **Ingress Controller**. Este recurso actúa como un balanceador de carga de capa 7 (HTTP/HTTPS), centralizando todo el tráfico exterior en una única IP pública y redirigiendo las peticiones a los servicios internos según la ruta (path) de la URL.

4.4.3.1 Manifiesto de Enrutamiento (Ingress)

Para aplicar esta nueva estructura, los servicios originales (ws-catalogo-service y ws-reservas-service) cambiaron su tipo de LoadBalancer a NodePort (o ClusterIP), volviéndose completamente invisibles desde el exterior. A continuación se muestra cómo quedó el servicio de catálogo modificado:

```

apiVersion: v1
kind: Service
metadata:
  name: ws-catalogo-service
spec:
  type: NodePort
  selector:
    app: ws-catalogo
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080

```

Después, se definió el recurso Ingress, que se encarga de dar las instrucciones al balanceador de Google Cloud para gestionar las reglas de rutas:

```

apiVersion: networking.k8s.io/v1
kind: Ingress

```

```
metadata:
  name: api-gateway-ingress-v2
spec:
  # Instruye a GKE para aprovisionar un balanceador HTTP de Google Cloud
  ingressClassName: "gce"
  rules:
    - http:
        paths:
          # Enrutamiento al microservicio de Catálogo
          - path: /eventos
            pathType: Prefix
            backend:
              service:
                name: ws-catalogo-service
                port:
                  number: 8080
          # Enrutamiento al microservicio de Reservas
          - path: /reservas
            pathType: Prefix
            backend:
              service:
                name: ws-reservas-service
                port:
                  number: 8080
```

4.4.3.2 Beneficios de la Arquitectura Resultante

La implementación de este manifiesto aporta tres mejoras muy claras a la red de la plataforma:

- **Reducción de costes:** Google Cloud ahora solo requiere un balanceador físico y una única IP pública, lo que reduce el gasto en infraestructura de red a la mitad.
- **Punto de acceso único:** Los usuarios o las aplicaciones cliente (como un frontend) no necesitan gestionar distintas direcciones IP. Todas las llamadas van al mismo sitio y solo cambia el sufijo de la URL (/eventos o /reservas).
- **Escalabilidad fácil:** Si en el futuro se añade un nuevo microservicio, basta con agregar su ruta en este archivo YAML, sin necesidad de crear más infraestructura pública en Google Cloud.

4.4.3.3 Ejecución y despliegue del Ingress Controller

Con los microservicios configurados en modo interno y tras añadir las anotaciones necesarias para habilitar los Grupos de Endpoints de Red (NEG) nativos de Google Cloud, se aplicó el recurso de enrutamiento unificado.

Como se puede observar en la **Figura 4-16**, los microservicios ya no muestran ninguna dirección en la columna EXTERNAL-IP, quedando totalmente protegidos dentro de la red privada del clúster.

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
kubernetes	ClusterIP	34.118.224.1	<none>	443/TCP
ws-catalogo-service	ClusterIP	34.118.231.126	<none>	8080/TCP
ws-reservas-service	ClusterIP	34.118.234.228	<none>	8080/TCP

Figura 4-16 Estado de los servicios del clúster tras la migración a topología ClusterIP

Una vez protegida la red interna, se ejecutó el comando para crear el objeto Ingress:

```
kubectl apply -f f2-kubernetes-cloud/ingress.yaml
```

Al utilizar el Ingress nativo de Google Kubernetes Engine (clase gce), el propio clúster se comunica directamente con la infraestructura de Google Cloud para aprovisionar un

balanceador de carga de capa 7, mapear las rutas HTTP y configurar automáticamente las reglas de reenvío.

Tras esperar unos minutos a que se propagara la configuración, se revisó el estado detallado del recurso con el comando de descripción:

Los resultados de la terminal se muestran en la **Figura 4-17**. En ella se comprueba que se ha asignado correctamente la IP pública global 8.232.193.87 y que, en la sección de backends, ambos microservicios han superado con éxito las comprobaciones de salud y están en estado HEALTHY.

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl describe ingress api-gateway-ingress-v2
Name:          api-gateway-ingress-v2
Labels:        <none>
Namespace:    default
Address:       8.232.193.87
Ingress Class: <none>
Default backend: <default>
Rules:
  Host      Path      Backends
  ----      -
  *
  /eventos  ws-catalogo-service:8080 (10.19.0.6:8080)
  /reservas ws-reservas-service:8080 (10.19.0.11:8080)
Annotations: ingress.kubernetes.io/backends:
              {"k8s1-8ded5be2-default-ws-catalogo-service-8080-dc2ac676":"HEALTHY","k8s1-8ded5be2-default-ws-reservas-service-8080-d8155e06":"HEALTHY",...
              ingress.kubernetes.io/forwarding-rule: k8s2-fr-s6h7yjdz-default-api-gateway-ingress-v2-d9tio0vs
              ingress.kubernetes.io/target-proxy: k8s2-tp-s6h7yjdz-default-api-gateway-ingress-v2-d9tio0vs
              ingress.kubernetes.io/url-map: k8s2-um-s6h7yjdz-default-api-gateway-ingress-v2-d9tio0vs
              kubernetes.io/ingress.class: gce
Events:
  Type      Reason      Age      From      Message
  ----      -
  Normal    Sync        8m33s    loadbalancer-controller  UrlMap "k8s2-um-s6h7yjdz-default-api-gateway-ingress-v2-d9tio0vs" created
  Normal    Sync        8m31s    loadbalancer-controller  TargetProxy "k8s2-tp-s6h7yjdz-default-api-gateway-ingress-v2-d9tio0vs" created
  Normal    Sync        8m14s    loadbalancer-controller  ForwardingRule "k8s2-fr-s6h7yjdz-default-api-gateway-ingress-v2-d9tio0vs" created
  Normal    IPChanged   8m13s    loadbalancer-controller  IP is now 8.232.193.87
  Normal    Sync        4m7s (x6 over 15m)    loadbalancer-controller  Scheduled for sync
```

Figura 4-17 Inspección detallada del Ingress Controller en GKE

Esta refactorización cambia por completo la forma de interactuar con la plataforma. Al haber eliminado los accesos públicos individuales, la documentación interactiva de Swagger UI se centraliza en la misma dirección IP pública asignada por el Ingress.

De este modo, las rutas directas para consultar y probar los contratos de los microservicios pasan a estar unificadas:

- **Swagger de eventos:** <http://8.232.193.87/eventos/swagger-ui/>
- **Swagger de reservas:** <http://8.232.193.87/reservas/swagger-ui/>

Esta prueba confirma el éxito de la topología: de cara al exterior, la plataforma actúa como un sistema único y compacto, mientras que internamente Kubernetes distribuye las peticiones de forma transparente a la capa interna de contenedores, garantizando la seguridad, el desacoplamiento y la eficiencia de costes.

4.5 Plan de Pruebas y Validación de la Arquitectura

4.5.1 Estrategia y Entorno de Pruebas

Una vez terminado el despliegue de la arquitectura en Google Kubernetes Engine (GKE) y configurado el punto de entrada único mediante Ingress, el siguiente paso fue realizar una serie de pruebas para validar el sistema. El objetivo aquí no es comprobar si la aplicación funciona a nivel de código, sino demostrar que la infraestructura es realmente robusta, elástica y tolerante a fallos.

Para que los resultados sean reales, las pruebas se han ejecutado directamente en el entorno de producción que montamos en el capítulo anterior, el cual utiliza la topología

interna ClusterIP, seguridad con Workload Identity y enrutamiento mediante el Ingress Controller.

La estrategia de validación se ha dividido en tres escenarios principales, que representan las mayores ventajas de utilizar un orquestador como Kubernetes:

1. Pruebas de Resiliencia y Autorrecuperación (Self-Healing): Consiste en simular la caída crítica de los servicios (eliminando Pods a la fuerza) para comprobar si el plano de control de Kubernetes es capaz de detectar el fallo mediante las sondas de salud (Liveness y Readiness) y de levantar una nueva instancia automáticamente, sin que tengamos que intervenir.

2. Pruebas de Actualización sin Corte de Servicio (Rolling Updates): Se demostrará cómo Kubernetes gestiona el despliegue de nuevas versiones o el reinicio de configuraciones sin generar ventanas de indisponibilidad (downtime). Para ello, se forzará la recarga de un microservicio y se verificará cómo el clúster orquesta la transición gradual entre los contenedores antiguos y los nuevos.

3. Pruebas de Escalabilidad Automática (Auto-Scaling): Consiste en evaluar cómo reacciona el clúster ante picos de tráfico masivos. Para ello, se configurará un escalador automático de Pods (Horizontal Pod Autoscaler o HPA) y se meterá carga artificial al sistema para obligar a Kubernetes a crear nuevas réplicas en tiempo real para absorber el tráfico.

Para realizar estas pruebas, se han utilizado las siguientes herramientas:

- **kubecti:** Es la herramienta de línea de comandos principal. La usaremos para provocar fallos en el clúster (al borrar Pods) y para monitorear eventos y el estado de los recursos en tiempo real.
- **Generadores de carga HTTP:** Herramientas de simulación de tráfico para lanzar miles de solicitudes concurrentes contra los microservicios. De esta forma, saturaremos el uso de CPU de los contenedores para comprobar si las reglas de autoescalado de GKE se disparan correctamente.

Con este marco de trabajo definido, en los siguientes apartados se detallan, paso a paso, cómo se ejecutó cada prueba y los resultados reales obtenidos en la terminal.

4.5.2 Pruebas de Resiliencia y Alta Disponibilidad (Self-Healing)

El objetivo de esta primera prueba es comprobar en la práctica una de las mayores ventajas de Kubernetes: su capacidad de autorrecuperación o self-healing. Tal como se explicó en los capítulos teóricos, los controladores (en este caso, el Deployment y su ReplicaSet) vigilan constantemente el clúster para asegurar que la infraestructura real coincida exactamente con lo especificado en los manifiestos YAML.

Para ello, se simulará la caída abrupta de un microservicio. En producción, esto suele deberse a un fallo en el hardware del servidor, a un error no controlado en el código o a un cierre forzado por falta de memoria RAM (Out Of Memory). En nuestro caso, forzaremos este fallo eliminando manualmente un Pod en servicio.

Lo que se espera es que el plano de control (Control Plane) detecte de inmediato que falta una réplica y levante automáticamente un Pod nuevo para recuperar la normalidad, sin que haya que hacer absolutamente nada manualmente.

La prueba consta de tres pasos:

1. Estado inicial: Comprobar que todos los Pods están levantados y en estado Running.

2. Inyección del fallo: Ejecutar el borrado forzoso de uno de los Pods.

3. Verificación: Observar en tiempo real cómo Kubernetes destruye el Pod afectado y crea su reemplazo.

Partimos de la siguiente situación inicial en el clúster:

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ws-catalogo-9d6d456f6-2pt68        2/2     Running   0           46h
ws-reservas-f9699b67-j2qsm         2/2     Running   0           46h
```

Figura 4-18 Estado inicial de los Pods del clúster

Una vez confirmada la estabilidad del sistema, se utiliza kubectl para borrar a la fuerza el Pod del catálogo (ws-catalogo-9d6d456f6-2pt68):

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl delete pod ws-catalogo-9d6d456f6-2pt68
pod "ws-catalogo-9d6d456f6-2pt68" deleted from default namespace
```

Figura 4-19 Inyección de fallo mediante la eliminación forzosa de la réplica

Rápidamente, se lanza el comando de consulta en modo de observación continua (-w) para ver cómo reacciona el Control Plane en tiempo real:

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods -w
NAME                                READY   STATUS    RESTARTS   AGE
ws-catalogo-9d6d456f6-2pt68        2/2     Running   0           46h
ws-reservas-f9699b67-j2qsm         2/2     Running   0           46h
ws-catalogo-9d6d456f6-2pt68        2/2     Terminating   0           46h
ws-catalogo-9d6d456f6-2pt68        2/2     Terminating   0           46h
ws-catalogo-9d6d456f6-pwl7k        0/2     Pending      0           0s
ws-catalogo-9d6d456f6-pwl7k        0/2     Pending      0           0s
ws-catalogo-9d6d456f6-pwl7k        0/2     Pending      0           0s
ws-catalogo-9d6d456f6-pwl7k        0/2     Pending      0           1s
ws-catalogo-9d6d456f6-pwl7k        0/2     ContainerCreating   0           1s
ws-catalogo-9d6d456f6-2pt68        0/2     Error        0           46h
ws-catalogo-9d6d456f6-2pt68        0/2     Error        0           46h
ws-catalogo-9d6d456f6-2pt68        0/2     Error        0           46h
ws-catalogo-9d6d456f6-pwl7k        1/2     Running      0           5s
ws-catalogo-9d6d456f6-pwl7k        1/2     Running      0           10s
ws-catalogo-9d6d456f6-pwl7k        2/2     Running      0           16s
ws-catalogo-9d6d456f6-pwl7k        2/2     Running      0           16s
```

Figura 4-20 Trazabilidad de la autorreparación (Self-Healing) del Pod ws-catalogo

Al analizar el registro de eventos anterior, se comprueba que el clúster reacciona exactamente como predice la teoría.

En el instante en que se envía la solicitud para borrar el Pod original (ws-catalogo-9d6d456f6-2pt68), su estado cambia a “Terminating”. Casi al mismo tiempo (en la marca de 0 segundos), el ReplicaSet se da cuenta de que debería haber un Pod en ejecución, pero no hay ninguno. Inmediatamente, se ordena crear un nuevo Pod (ws-catalogo-9d6d456f6-pwl7k), que entra en estado Pending a la espera de que se le asigne un nodo físico para trabajar.

La trazabilidad de los logs muestra unos tiempos de recuperación sumamente ágiles:

- **En el segundo 1:** El nuevo Pod pasa a ContainerCreating, momento en el que descarga las imágenes y prepara la creación de los contenedores.
- **En el segundo 5:** Arranca el primero de los dos contenedores del Pod (1/2) y entra en estado Running.
- **En el segundo 16:** El Pod está totalmente operativo (2/2), habiendo arrancado el proxy a la base de datos y superado con éxito las sondas de disponibilidad de Quarkus (Readiness Probes).

Por último, se realiza una comprobación estática del clúster para obtener el resultado final. El nuevo Pod de catálogo está funcionando con normalidad, asumiendo el tráfico, mientras que el Pod de reservas (ws-reservas-f9699b67-j2qsm) sigue intacto y registra un tiempo de actividad ininterrumpido de 46 horas. Esto demuestra que la caída y recuperación de un microservicio están totalmente aisladas y no afectan al resto del sistema.

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ws-catalogo-9d6d456f6-pwl7k        2/2     Running   0           32s
ws-reservas-f9699b67-j2qsm         2/2     Running   0           46h
```

Figura 4-21 Estado final de los Pods del clúster

Como conclusión, queda demostrado que la plataforma montada en GKE es tolerante a fallos. Si un servicio se cae en un entorno de producción real, Kubernetes aplica sus mecanismos de autorrecuperación para restaurar la alta disponibilidad de forma autónoma, garantizando el servicio sin requerir la intervención de un administrador de sistemas.

4.5.3 Pruebas de Actualización sin Corte de Servicio (Rolling Updates)

Desplegar una nueva versión de una aplicación tradicionalmente implicaba cortes de servicio o programar ventanas de mantenimiento de madrugada. Kubernetes soluciona este problema mediante la estrategia de actualización continua (Rolling Update), que viene configurada por defecto en los Deployments.

El objetivo de esta prueba es comprobar en la práctica cómo el clúster gestiona el lanzamiento de una nueva versión o el reinicio de los contenedores. Al aplicar un cambio (como actualizar la imagen de Docker a la versión v2 o forzar un reinicio), Kubernetes no tumba los Pods antiguos de golpe. En su lugar, realiza una transición progresiva: levanta un Pod nuevo, espera a que esté 100% operativo y conectado a la base de datos, y solo entonces elimina el Pod de la versión anterior.

Para comprobarlo, se va a forzar un reinicio del microservicio de catálogo (ws-catalogo). El estado de los Pods antes de empezar la prueba es el siguiente:

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ws-catalogo-9d6d456f6-hlz2m        2/2     Running   0           27s
ws-reservas-f9699b67-bmm4w         2/2     Running   0           64s
```

Figura 4-22 Estado inicial de los Pods del clúster

Una vez comprobada la estabilidad, se lanza el comando de reinicio sobre el Deployment:

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl rollout restart deployment ws-catalogo
deployment.apps/ws-catalogo restarted
```

Figura 4-23 Ejecución del comando de reinicio progresivo del Deployment

Inmediatamente después, se observa el estado de los recursos en la terminal para capturar el proceso de sustitución de los Pods en tiempo real:

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
ws-catalogo-9d6d456f6-hlz2m	2/2	Running	0	87s
ws-reservas-f9699b67-bmm4w	2/2	Running	0	2m4s
ws-catalogo-56cc65745f-cz7zg	0/2	Pending	0	0s
ws-catalogo-56cc65745f-cz7zg	0/2	Pending	0	0s
ws-catalogo-56cc65745f-cz7zg	0/2	Pending	0	0s
ws-catalogo-56cc65745f-cz7zg	0/2	Pending	0	0s
ws-catalogo-56cc65745f-cz7zg	0/2	ContainerCreating	0	0s
ws-catalogo-56cc65745f-cz7zg	1/2	Running	0	3s
ws-catalogo-56cc65745f-cz7zg	2/2	Running	0	15s
ws-catalogo-56cc65745f-cz7zg	2/2	Running	0	17s
ws-catalogo-56cc65745f-cz7zg	2/2	Running	0	17s
ws-catalogo-9d6d456f6-hlz2m	2/2	Terminating	0	2m46s
ws-catalogo-9d6d456f6-hlz2m	2/2	Terminating	0	2m46s
ws-catalogo-9d6d456f6-hlz2m	0/2	Error	0	2m47s
ws-catalogo-9d6d456f6-hlz2m	0/2	Error	0	2m47s
ws-catalogo-9d6d456f6-hlz2m	0/2	Error	0	2m47s

Figura 4-24 Trazabilidad del proceso de sustitución de los Pods en tiempo real

Al analizar este registro de eventos, se comprueba que el Rolling Update funciona como un mecanismo de seguridad para garantizar que la plataforma nunca deje de responder. El flujo real que orquestó el controlador fue el siguiente:

1. Aprovisionamiento previo (Segundos 0 - 3): En lugar de borrar el Pod que estaba dando servicio (ws-catalogo-9d6d456f6-hlz2m), el clúster lo mantiene en funcionamiento y empieza a levantar un Pod nuevo (ws-catalogo-56cc65745f-cz7zg). Se puede notar que pertenecen a ReplicaSets distintos porque cambia el hash del nombre del Pod. Esta nueva instancia pasa por los estados Pending y ContainerCreating.

2. Arranque seguro (Segundo 15): El nuevo Pod arranca por completo (2/2 contenedores listos). En este instante, Kubernetes valida que la instancia está viva, que el proxy Sidecar se ha conectado a Cloud SQL y que la aplicación está lista para recibir tráfico HTTP.

3. Drenaje y terminación (Minuto 2:46): Exclusivamente después de que el Pod nuevo ha superado sus sondas de salud (Readiness Probes), el clúster actualiza el balanceador interno para redirigir el tráfico a él y ordena apagar de forma controlada (Terminating) el Pod antiguo.

Una última comprobación estática del clúster confirma que la operación ha sido un éxito. El nuevo Pod de catálogo ya está atendiendo las peticiones con normalidad, mientras que el microservicio de reservas (ws-reservas) no ha sufrido ninguna alteración y mantiene su tiempo de actividad intacto.

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ws-catalogo-56cc65745f-cz7zg	2/2	Running	0	29s
ws-reservas-f9699b67-bmm4w	2/2	Running	0	3m35s

Figura 4-25 Estado final estabilizado tras la actualización

Como conclusión, esta prueba constata que la infraestructura construida permite al equipo desplegar nuevas versiones de la aplicación, aplicar parches de seguridad o forzar reinicios de configuración en pleno horario de máxima concurrencia, sin provocar caídas ni ventanas de indisponibilidad (Zero Downtime) que afecten a los usuarios finales.

4.5.4 Pruebas de escalabilidad Automática (Autoscaling)

El último escenario de validación tiene como objetivo demostrar la capacidad de respuesta de la arquitectura ante picos de demanda imprevistos. En un entorno tradicional, absorber un aumento repentino del tráfico requiere un sobredimensionamiento manual previo de los servidores, lo que genera costes fijos innecesarios cuando el sistema está en reposo.

Kubernetes resuelve esto de forma nativa mediante el componente Horizontal Pod Autoscaler (HPA).

Para implementar el autoescalado en Google Kubernetes Engine (GKE), no es necesario aprovisionar infraestructura adicional. El clúster incluye por defecto el Metrics Server, un componente del plano de control encargado de monitorizar en tiempo real el uso de CPU y memoria de los contenedores.

El único requisito arquitectónico previo para que el HPA funcione correctamente es que los microservicios tengan definidos sus límites y peticiones de recursos (`resources.requests`) en su manifiesto de despliegue original. Si el contenedor no declara al clúster cuánta CPU necesita para funcionar en condiciones normales, el HPA no tiene una base matemática sobre la que calcular los porcentajes de saturación y el autoescalado no se ejecutará.

4.5.4.1 Configuración del Autoescalador (HPA)

Una vez asegurado de que el microservicio de catálogo tiene definidas sus peticiones base de CPU, se diseñó un manifiesto declarativo para establecer las reglas del escalado.

El siguiente archivo YAML instruye al clúster a mantener un mínimo de 1 réplica en momentos de bajo tráfico para optimizar recursos, otorgándole la autonomía de escalar hasta un máximo de 5 réplicas bajo demanda. Como ajuste específico para esta arquitectura, el umbral de escalado (`averageUtilization`) se ha configurado excepcionalmente en el **400%**. Esta configuración tolerante permite al sistema absorber los picos naturales de CPU que genera el framework de Java durante el arranque de la aplicación (los cuales superan el 100% de la petición base asignada), evitando falsos positivos en el escalado, pero reaccionando de forma agresiva y eficaz ante picos de tráfico real sostenidos. No obstante, se es consciente de que las buenas prácticas de producción para máquinas virtuales pesadas (JVM) requieren implementar una `startupProbe`. Esta sonda especializada delegaría la comprobación inicial del arranque, lo que permitiría en un futuro configurar este HPA con umbrales de CPU más estrictos (70%-80%) y evitaría distorsiones en las métricas de autoescalado.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: ws-catalogo-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: ws-catalogo
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 400
```

Tras aplicar este manifiesto con el comando `kubectl apply`, el controlador del HPA entra en funcionamiento y comienza a supervisar la carga de trabajo del microservicio.

4.5.4.2 Ejecución de la Prueba de Carga

Para comprobar que el autoescalado horizontal (HPA) funciona correctamente bajo estrés, realizamos una prueba de carga continua en el microservicio de catálogo. Antes de iniciar la ejecución, se volvió a desplegar el Ingress del clúster, lo que nos asignó una nueva IP pública (34.120.13.33) que utilizamos como objetivo para enviar el tráfico.

Para generar esta simulación de tráfico utilizamos la herramienta ApacheBench (ab). Lanzamos un volumen de 100.000 peticiones con una concurrencia de 15 usuarios simultáneos, añadiendo el parámetro Keep-Alive para mantener la conexión viva, de modo que el cortafuegos de Google Cloud no nos bloquee la prueba por defecto.

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % ab -k -n 100000 -c 15 http://34.120.13.33/eventos/
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking 34.120.13.33 (be patient)
Completed 10000 requests
Completed 20000 requests
Completed 30000 requests
Completed 40000 requests
Completed 50000 requests
Completed 60000 requests
Completed 70000 requests
Completed 80000 requests
```

Figura 4-26 Ejecución del comando de prueba de carga con ApacheBench hacia la nueva IP pública

En cuanto empezó a circular el tráfico, monitorizamos en tiempo real el comportamiento del HPA. Partíamos de un escenario tranquilo en el que el Pod solo consumía el uso básico de Java (un 45% de los recursos mínimos asignados). Sin embargo, al recibir el impacto de las peticiones, la CPU se disparó bruscamente, alcanzando picos superiores al 3700%.

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 45%/400%	1	5	1	2m6s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 46%/400%	1	5	1	2m18s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 3760%/400%	1	5	1	2m35s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 3760%/400%	1	5	4	2m45s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 4544%/400%	1	5	5	2m50s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 4363%/400%	1	5	5	3m35s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 3346%/400%	1	5	5	3m45s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 3699%/400%	1	5	5	3m49s
ws-catalogo-hpa	Deployment/ws-catalogo	cpu: 2894%/400%	1	5	5	4m4s

Figura 4-27 Monitorización en vivo del HPA reaccionando a la subida de CPU

Como se refleja en la **Figura 5-10**, Kubernetes detectó la saturación al instante. Al superar el límite configurado, el autoescalador reaccionó levantando más instancias de forma escalonada: primero subió a 4 réplicas y, segundos después, llegó al tope máximo de 5 réplicas que habíamos marcado como límite en la arquitectura.

Mientras el HPA daba las órdenes, comprobamos cómo se levantaban físicamente esos contenedores.

```
administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
ws-catalogo-76b68d8ffb-qtc44	2/2	Running	0	2m31s
ws-reservas-f9699b67-5pqkl	2/2	Running	0	115m
ws-catalogo-76b68d8ffb-4c67v	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-4c67v	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-4c67v	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-5npcn	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-t8vr9	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-5npcn	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-4c67v	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-4c67v	0/2	ContainerCreating	0	0s
ws-catalogo-76b68d8ffb-t8vr9	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-5npcn	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-t8vr9	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-5npcn	0/2	ContainerCreating	0	0s
ws-catalogo-76b68d8ffb-t8vr9	0/2	ContainerCreating	0	0s
ws-catalogo-76b68d8ffb-5npcn	1/2	Running	0	3s
ws-catalogo-76b68d8ffb-t8vr9	1/2	Running	0	3s
ws-catalogo-76b68d8ffb-4c67v	1/2	Running	0	3s
ws-catalogo-76b68d8ffb-rvf49	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-rvf49	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-rvf49	0/2	Pending	0	0s
ws-catalogo-76b68d8ffb-rvf49	0/2	ContainerCreating	0	0s
ws-catalogo-76b68d8ffb-rvf49	1/2	Running	0	2s
ws-catalogo-76b68d8ffb-5npcn	1/2	Running	0	28s
ws-catalogo-76b68d8ffb-5npcn	2/2	Running	0	34s
ws-catalogo-76b68d8ffb-t8vr9	2/2	Running	0	35s
ws-catalogo-76b68d8ffb-4c67v	2/2	Running	0	35s
ws-catalogo-76b68d8ffb-t8vr9	2/2	Running	0	38s
ws-catalogo-76b68d8ffb-t8vr9	2/2	Running	0	38s
ws-catalogo-76b68d8ffb-4c67v	2/2	Running	0	39s
ws-catalogo-76b68d8ffb-rvf49	1/2	Running	0	29s
ws-catalogo-76b68d8ffb-4c67v	2/2	Running	0	39s
ws-catalogo-76b68d8ffb-rvf49	2/2	Running	0	33s

Figura 4-28 Creación secuencial de las nuevas réplicas en el nodo de Google Cloud

En la **Figura 5-11** se ve cómo los nuevos Pods pasaron rapidísimo de estar pendientes a ejecutarse. En menos de 40 segundos, todos los contenedores arrancaron, se conectaron a la base de datos a través del proxy y pasaron los controles de salud, listos para trabajar.

Al final de la prueba, el sistema consiguió estabilizarse por completo con sus 5 réplicas funcionando a pleno rendimiento y absorbiendo el ataque sin que se cayera el servicio.

Es importante destacar que durante este escalado masivo a múltiples réplicas, la estabilidad de la base de datos de Cloud SQL se mantuvo intacta gracias a la gestión del pool de conexiones. En lugar de abrir una conexión nueva por cada petición entrante, el framework Quarkus gestiona de forma nativa un pool de conexiones mediante la extensión Agroal. Al dimensionar correctamente este pool en el `application.properties` de cada microservicio, se garantiza que, incluso con el HPA al máximo de réplicas, todas las conexiones concurrentes no superen nunca el límite máximo tolerado por PostgreSQL. Esto evita escenarios catastróficos, como el agotamiento de conexiones (Connection Exhaustion), y elimina la necesidad de integrar proxies externos, como PgBouncer.

```

administrador@MacBook-Pro-de-Victor-Garcia gestion-eventos % kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
ws-catalogo-76b68d8ffb-4c67v       2/2     Running   0           52s
ws-catalogo-76b68d8ffb-5npcn       2/2     Running   0           52s
ws-catalogo-76b68d8ffb-qtc44       2/2     Running   0           7m33s
ws-catalogo-76b68d8ffb-rvf49       2/2     Running   0           42s
ws-catalogo-76b68d8ffb-t8vr9       2/2     Running   0           52s
ws-reservas-f9699b67-5pqkL         2/2     Running   0           120m

```

Figura 4-29 Estado final del clúster estabilizado con 5 réplicas

4.5.4.3 Evaluación de Resultados

Con esta configuración y tras los resultados obtenidos en la prueba de carga, la arquitectura del sistema demuestra alcanzar un equilibrio óptimo entre rendimiento y eficiencia de costes. Por un lado, garantiza la resiliencia y la alta disponibilidad del servicio ante picos inesperados de demanda, reaccionando automáticamente para absorber el tráfico. Por otro lado, al desescalar a una única réplica durante los periodos de inactividad, se minimiza el consumo de recursos físicos en el clúster, lo que se traduce en un ahorro directo en la facturación de la nube. Todo esto confirma que el diseño implementado es robusto, elástico y cumple con los requisitos de un entorno de producción real.

5. Conclusiones y Trabajos Futuros

5.1 Objetivos alcanzados

Tras finalizar la implementación y el plan de validación de la arquitectura, se han cumplido satisfactoriamente los objetivos planteados al inicio del proyecto:

- **Diseño e implementación de arquitectura cloud-native:** Se ha completado con éxito la migración de un entorno de microservicios a un clúster de producción en Google Kubernetes Engine (GKE), lo que ha permitido lograr una infraestructura modular, escalable y totalmente desacoplada de la infraestructura física subyacente.
- **Orquestación eficiente y declarativa:** Se ha implementado una gestión automatizada de la infraestructura mediante manifiestos declarativos. Esto permite que el estado del sistema sea consistente y reproducible, automatizando componentes críticos como el balanceo de tráfico mediante controladores Ingress y el almacenamiento persistente de datos mediante Cloud SQL.
- **Garantía de resiliencia y alta disponibilidad:** Se ha validado la capacidad de autorreparación (self-healing) del sistema, comprobando cómo Kubernetes recupera automáticamente los servicios ante fallos inesperados. Asimismo, se han realizado despliegues exitosos con actualizaciones sin interrupciones (rolling updates), lo que ha asegurado la continuidad del servicio a lo largo del ciclo de vida de la aplicación.
- **Validación de la elasticidad del sistema:** Se ha configurado y probado con éxito el Horizontal Pod Autoscaler (HPA). Mediante pruebas de carga reales, se ha verificado que la arquitectura responde dinámicamente a las variaciones de la demanda, optimizando el uso de los recursos físicos del clúster y minimizando los costes operativos en periodos de baja actividad.

5.2 Valoración técnica del proyecto

El desarrollo de este trabajo arroja un balance global muy positivo y demuestra las ventajas del paradigma cloud-native. Sin embargo, también ha supuesto la complejidad operativa de desplegar y mantener sistemas distribuidos. A continuación, se exponen las conclusiones técnicas más destacadas de la implementación:

- **Gestión dinámica y eficiencia económica:** La combinación de Kubernetes y el HPA se ha demostrado una herramienta invaluable. La capacidad del clúster para adaptar sus

réplicas en tiempo real evita el sobreaprovisionamiento clásico (mantener servidores encendidos sin carga real) y garantiza la estabilidad cuando el tráfico se dispara, optimizando directamente la facturación en la nube.

- **El desafío en la gestión de recursos de CPU:** Encontrar el equilibrio perfecto al configurar los requests y los limits en los manifiestos ha supuesto un reto importante. Al tratarse de una aplicación Java, el contenedor requiere un esfuerzo extra de CPU solo para arrancar. Se comprobó empíricamente que ignorar este comportamiento provoca que el HPA escale réplicas por error o que el servidor se sature, lo que deja claro que en Kubernetes no basta con desplegar, sino que hay que perfilar el consumo real del software.
- **Tiempos de aprovisionamiento asíncronos (El caso de Ingress):** Durante los despliegues de red, se constató un comportamiento crítico de GKE: la creación del controlador Ingress no es instantánea. Al depender del aprovisionamiento de un balanceador de Carga físico de Google Cloud (GCLB) y de la asignación de una IP pública, el proceso demora varios minutos. Durante este periodo de propagación y estabilización de los health checks, el clúster puede devolver errores temporales (como 404 o 502), lo que obliga a diseñar estrategias de despliegue que contemplen esta asincronía para evitar interrupciones de servicio.
- **Infraestructura declarativa y observabilidad:** Definir toda la arquitectura mediante manifiestos YAML garantiza que el entorno sea reproducible y esté documentado. La integración con los servicios de Google Cloud (como la conexión a PostgreSQL mediante Cloud SQL Proxy) ha demostrado ser robusta. No obstante, operar el clúster únicamente desde la línea de comandos presenta limitaciones. A medida que el sistema crece, resulta indispensable integrar plataformas visuales de monitorización para detectar anomalías de manera proactiva.

5.3 Trabajos Futuros

Aunque la arquitectura desplegada cumple con los objetivos de escalabilidad, resiliencia y alta disponibilidad planteados en este proyecto, el ecosistema cloud-native ofrece un amplio margen de mejora. Como líneas de trabajo futuras para evolucionar y optimizar este sistema, se proponen las siguientes implementaciones:

5.3.1 Integración avanzada de Inteligencia Artificial mediante Model Context Protocol (MCP)

Como evolución natural hacia la automatización inteligente y las operaciones asistidas por IA (AIOps), se propone desplegar un servidor MCP nativo en el clúster de GKE. Este protocolo estándar actuaría como un puente bidireccional y seguro entre grandes modelos de lenguaje (LLMs) y la API de Kubernetes, dotando a la IA del contexto real de la infraestructura. Esta integración habilitaría casos de uso disruptivos para la arquitectura actual, tales como:

- **Resolución de incidencias (Troubleshooting) asistida:** En lugar de depender de la ejecución manual de comandos kubectl para investigar un problema, el modelo podría consultar de forma autónoma los logs de los contenedores Quarkus, los eventos del sistema y las métricas de consumo. Ante un fallo, la IA analizaría el contexto completo y propondría, o incluso ejecutaría si tiene permisos, la remediación exacta.
- **Autoescalado predictivo e inteligente:** Más allá de la respuesta reactiva del Horizontal Pod Autoscaler (HPA) configurado en este proyecto, la IA podría utilizar MCP para analizar patrones históricos de tráfico y sugerir ajustes dinámicos en los parámetros

de requests y limits. Esto permitiría al clúster adelantarse a los picos de carga antes de que se produzcan.

- **Auditoría de seguridad y políticas de red:** El modelo podría examinar continuamente los manifiestos aplicados y la configuración de red interna del clúster, detectando en tiempo real vulnerabilidades, permisos excesivos o ineficiencias. Con esta implementación, la plataforma daría un salto cualitativo. Pasaría de ser una infraestructura puramente declarativa a un ecosistema "consciente", capaz de asistir activamente al ingeniero DevOps en la toma de decisiones operativas.

5.3.2 FinOps y Sostenibilidad (Green Computing)

Tras demostrar que el autoescalado optimiza los recursos, el siguiente paso es medir con precisión su impacto económico y medioambiental. Se propone la integración de herramientas como Kubecost o Kepler para monitorizar en tiempo real el coste de facturación de Google Cloud y la huella de carbono generada por cada microservicio. Esto permitiría afinar aún más los requests y los límites de CPU de los Pods, buscando no solo el rendimiento técnico, sino también la eficiencia económica y la sostenibilidad del sistema.

Si bien las decisiones arquitectónicas de este proyecto (como el autoescalado dinámico o el aislamiento lógico de las bases de datos) persiguen la optimización de recursos, la elaboración de un modelo numérico exacto de costes operativos (Total Cost of Ownership - TCO) queda fuera del alcance inicial.

5.3.3 Evolución hacia una Malla de Servicios (Service Mesh)

A medida que la arquitectura crezca añadiendo nuevos microservicios que acompañen al backend actual, la comunicación interna se volverá más compleja. La adopción de una tecnología como Istio permitiría desacoplar la lógica de red del código de la aplicación. Esto aportaría seguridad por defecto mediante cifrado de extremo a extremo (mTLS) entre los Pods y habilitaría estrategias de despliegue avanzadas, como los Canary Releases (despliegues canarios), que permiten probar nuevas versiones del código con un porcentaje mínimo de usuarios antes de su despliegue total.

5.3.4 Automatización total del despliegue mediante GitOps

Actualmente, la aplicación de los manifiestos YAML en el clúster requiere procesos manuales. Se propone migrar la gestión de la infraestructura a un modelo GitOps mediante herramientas como ArgoCD. De este modo, el repositorio Git se convertiría en la única fuente de verdad: cualquier cambio aprobado en el código fuente o en la configuración de la base de datos desencadenaría automáticamente la creación de una nueva imagen de Docker y su sincronización inmediata en GKE, cerrando por completo el ciclo de Integración y Despliegue Continuo (CI/CD) sin intervención humana.

5.4 Reflexión Final

La realización de este Trabajo Fin de Grado ha supuesto un punto de inflexión en mi formación, marcando la transición definitiva entre la etapa académica y la realidad de la industria. Aunque durante la carrera he consolidado una base teórica sólida en ingeniería informática, este proyecto ha sido la oportunidad de aplicar esos conocimientos en un escenario complejo, asumiendo el reto de diseñar, desplegar y gestionar una infraestructura cloud-native completa.

A nivel personal, este trabajo ha sido un ejercicio de perseverancia. Enfrentarme a las dificultades técnicas que plantea un entorno como Google Kubernetes Engine (GKE), depurar manifiestos o ajustar el comportamiento de la JVM de Java en contenedores, me ha obligado a desarrollar una mentalidad mucho más crítica. He aprendido que la ingeniería no termina con la escritura del código. La verdadera responsabilidad comienza por entender cómo ese software interactúa con la infraestructura, cómo escala, cómo se asegura y cómo se comporta ante fallos imprevistos.

He interiorizado la cultura DevOps no solo como un concepto teórico, sino también como una metodología imprescindible para la operatividad de los sistemas. La experiencia de ver cómo la arquitectura diseñada es capaz de autorrepararse o de adaptarse de forma autónoma ante picos de tráfico me ha dado una perspectiva muy clara del valor del paradigma cloud-native. Ha sido enormemente gratificante ver cómo, tras muchas pruebas y ajustes, el sistema ha dejado de ser un conjunto de piezas aisladas para convertirse en un ecosistema operativo sólido.

En definitiva, este TFG no representa solo el cierre de mi etapa universitaria, sino también el inicio de mi trayectoria profesional. Me llevo un criterio técnico mucho más maduro, la experiencia práctica necesaria para afrontar sistemas distribuidos complejos y, sobre todo, la confianza de saber que tengo las herramientas para resolver los retos técnicos que demanda el mercado actual.

Bibliografía

1. **Fowler, Martin.** *martinfowler.com*. [En línea] 25 de Marzo de 2014. <https://martinfowler.com/articles/microservices.html>.
2. **Amazon Web Services.** What are Microservices? *aws.amazon.com*. [En línea] 2025. <https://aws.amazon.com/es/microservices/>.
3. **IBM Cloud Education.** ¿Qué es la contenerización? *ibm.com*. [En línea] 2024. <https://www.ibm.com/es-es/topics/containerization>.
4. **Docker Inc.** What is a Container? *docker.com*. [En línea] 2025. <https://www.docker.com/resources/what-container/>.
5. **Red Hat.** ¿Qué es la orquestación de contenedores? *redhat.com*. [En línea] 2025. <https://www.redhat.com/es/topics/containers/what-is-container-orchestration>.
6. **The Kubernetes Authors.** Overview. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/overview/>.
7. **Borg, Omega, and Kubernetes. Burns, Brendan, y otros.** 1, New York : ACM, 2016, Vol. 13.
8. **The Kubernetes Authors.** Declarative Management of Kubernetes Objects Using Configuration Files. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/tasks/manage-kubernetes-objects/declarative-config/>.
9. —. Controllers. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/architecture/controller/>.
10. *Kubernetes and the Path to Cloud Native.* **Brewer, Eric A.** s.l. : ACM, 2015.
11. **Morris, Kief.** *Infrastructure as Code: Dynamic Systems for the Cloud Age*. Sebastopol : O'Reilly Media, 2020.
12. **The Kubernetes Authors.** Kubernetes Components. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/overview/components/>.
13. —. Node Components. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/overview/components/#node-components>.
14. —. Pods. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/workloads/pods/>.
15. —. Sidecar Containers. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/>.
16. —. ReplicaSet. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/>.
17. —. Deployments. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>.
18. —. Service. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/services-networking/service/>.
19. —. Ingress. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/services-networking/ingress/>.

20. —. Persistent Volumes. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/storage/persistent-volumes/>.
21. —. Storage Classes. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/storage/storage-classes/>.
22. —. ConfigMaps. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/configuration/configmap/>.
23. —. Secrets. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/configuration/secret/>.
24. **Cloud Native Computing Foundation (CNCF)**. Managed Kubernetes. *Cloud Native Glossary*. [En línea] 2026. <https://glossary.cncf.io/kubernetes/>.
25. **The Kubernetes Authors**. Turnkey Cloud Solutions. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/setup/production-environment/>.
26. **Docker Authors**. Swarm mode key concepts. *Docker Documentation*. [En línea] 2026. <https://docs.docker.com/engine/swarm/key-concepts/>.
27. **The Kubernetes Authors**. Understanding Kubernetes Objects. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/overview/working-with-objects/kubernetes-objects/>.
28. **Docker Authors**. Deploy a service to a swarm. *Docker Documentation*. [En línea] 2026. <https://docs.docker.com/engine/swarm/how-swarm-mode-works/services/>.
29. **The Kubernetes Authors**. Horizontal Pod Autoscaling. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>.
30. **Docker Authors**. Administer and maintain a swarm of Docker Engines. *Docker Documentation*. [En línea] 2026. https://docs.docker.com/engine/swarm/admin_guide/.
31. **The Kubernetes Authors**. Configure Liveness, Readiness and Startup Probes. *Kubernetes Documentation*. [En línea] <https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>.
32. **Docker Authors**. Use overlay networks. *Docker Documentation*. [En línea] 2026. <https://docs.docker.com/engine/network/drivers/overlay/>.
33. **The Kubernetes Authors**. Cluster Networking. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/cluster-administration/networking/>.
34. **Docker Authors**. Manage data in Docker. *Docker Documentation*. [En línea] 2026. <https://docs.docker.com/engine/storage/>.
35. **The Kubernetes Authors**. Storage. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/concepts/storage/>.
36. **Cloud Native Computing Foundation (CNCF)**. Software Conformance. *CNCF Projects*. [En línea] 2026. <https://www.cncf.io/projects/>.
37. **The Kubernetes Authors**. Extend the Kubernetes API with CustomResourceDefinitions. *Kubernetes Documentation*. [En línea] 2026. <https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/>.
38. **Docker Authors**. Swarm mode overview. *Docker Documentation*. [En línea] 2026. <https://docs.docker.com/engine/swarm/>.

39. **Cloud Native Computing Foundation (CNCF)**. CNCF Annual Survey. *CNCF Reports*. [En línea] 2026. <https://www.cncf.io/reports/>.